

Design and Analysis of Algorithms

Lecture 1 Introductions



Algorithm definition

- An **algorithm** a computational **procedure** that takes an **input** and produces an **output** in order to solve a well-specified computational problem.
 - 1. The instructions must be exact enough to be implemented mechanically;
 - 2. The number of instructions must be finite.
- Input, output,
- Computational problem, instance of a problem
- Correct, efficient, precise, mechanical,



算法 ALGORITHM

是在有限步骤内求解某一问题所使用的一组定义明确的规则。

一个算法应该具有以下五个重要的特征：

1. **有穷性：** 一个算法必须保证执行有限步之后结束；
2. **确切性：** 算法的每一步骤必须有确切的定义；
3. **输 入：** 一个算法有0个或多个输入，以刻画运算对象的初始情况，所谓0个输入是指算法本身限定了初始条件；
4. **输 出：** 一个算法有一个或多个输出，以反映对输入数据加工后的结果。没有输出的算法是毫无意义的；
5. **可行性：** 算法原则上能够精确地运行，而且人们用笔和纸做有限次运算后即可完成。



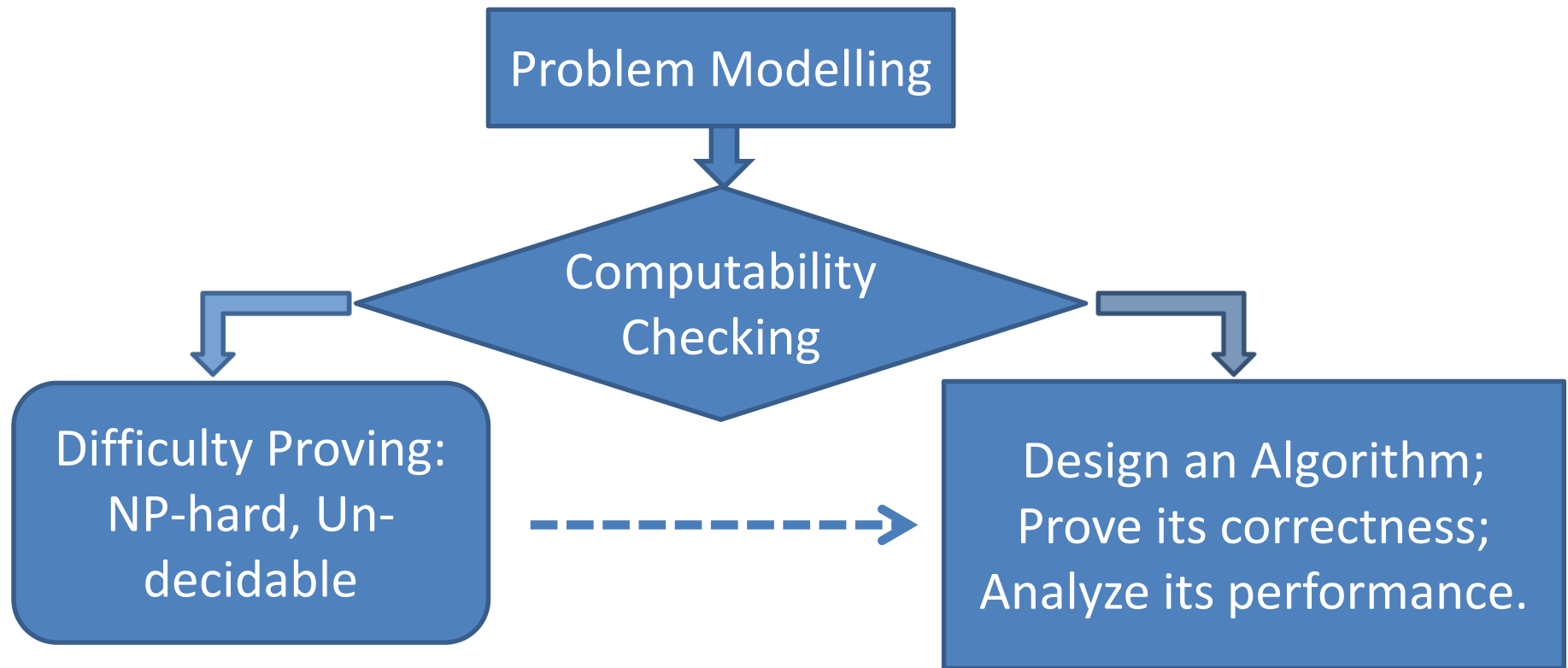
Example: Sorting problem

The sorting problem

- **Input:** a sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- **Output:** a permutation $\langle a_1', a_2', \dots, a_n' \rangle$, s.t. $a_1' \leq a_2' \leq \dots \leq a_n'$
- An **instance** of a problem:
 - the input needed to compute a solution (e.g.: $\langle 5, 3, 6, 2 \rangle$)
- An algorithm is **correct** if it ends with the correct output in a finite amount of time, on any legitimate input



A general framework



Classification of Algorithms

- By problem types

- Sorting
- Searching
- String processing
- Graph problems
- Combinatorial problems
- Geometric problems
- Numerical problems

- By design paradigms

- Divide-and-conquer
- Incremental
- Dynamic programming
- Greedy algorithms
- Randomized/probabilistic



Order of growth

- *Alg.:* MIN ($a[1], \dots, a[n]$)

$m \leftarrow a[1];$

for $i \leftarrow 2$ to n

if $a[i] < m$

then $m \leftarrow a[i];$

- Running time:

$T(n) = 1$ [first step] + (n) [for loop] + $(n-1)$ [if condition] +
 $(n-1)$ [the assignment in then] = $3n - 1$

- Order (rate) of growth:

- The leading term of the formula
- Gives a simple characterization of the algorithm's efficiency
- Expresses the asymptotic behavior of the algorithm

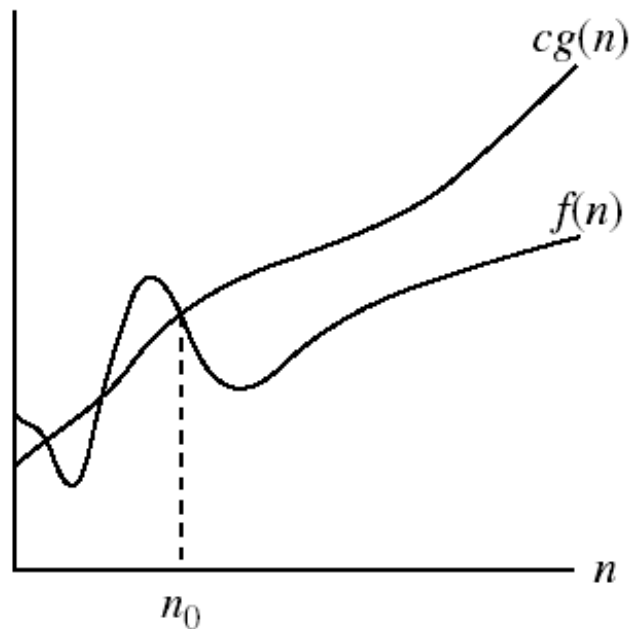
- $T(n)$ grows like n



Asymptotic notations

- *O-notation*

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$



- Intuitively: $O(g(n))$ = the set of functions with a smaller or same order of growth as $g(n)$

$g(n)$ is an *asymptotic upper bound* for $f(n)$.



Examples

- $2n^2 = O(n^3)$: $2n^2 \leq cn^3 \Rightarrow 2 \leq cn \Rightarrow c = 1$ and $n_0 = 2$

- $n^2 = O(n^2)$: $n^2 \leq cn^2 \Rightarrow c \geq 1 \Rightarrow c = 1$ and $n_0 = 1$

- $1000n^2 + 1000n = O(n^2)$:

$$1000n^2 + 1000n \leq cn^2 \leq cn^2 + 1000n \Rightarrow c = 1001 \text{ and } n_0 = 1$$

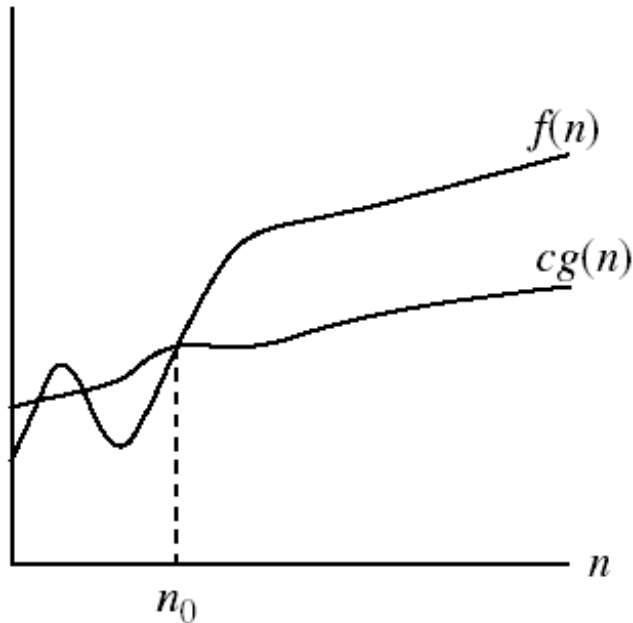
- $n = O(n^2)$: $n \leq cn^2 \Rightarrow cn \geq 1 \Rightarrow c = 1$ and $n_0 = 1$



Asymptotic notations (cont.)

- Ω - notation

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$.



- Intuitively: $\Omega(g(n))$ = the set of functions with a larger or same order of growth as $g(n)$

$g(n)$ is an *asymptotic lower bound* for $f(n)$.



Examples

– $5n^2 = \Omega(n)$

$\exists c, n_0$ such that: $0 \leq cn \leq 5n^2 \Rightarrow cn \leq 5n^2 \Rightarrow c = 1$ and $n_0 = 1$

– $100n + 5 \neq \Omega(n^2)$

$\exists c, n_0$ such that: $0 \leq cn^2 \leq 100n + 5$

$$100n + 5 \leq 100n + 5n \quad (\forall n \geq 1) = 105n$$

$$cn^2 \leq 105n \Rightarrow n(cn - 105) \leq 0$$

Since n is positive $\Rightarrow cn - 105 \leq 0 \Rightarrow n \leq 105/c$

$\Rightarrow$ contradiction: n cannot be smaller than a constant

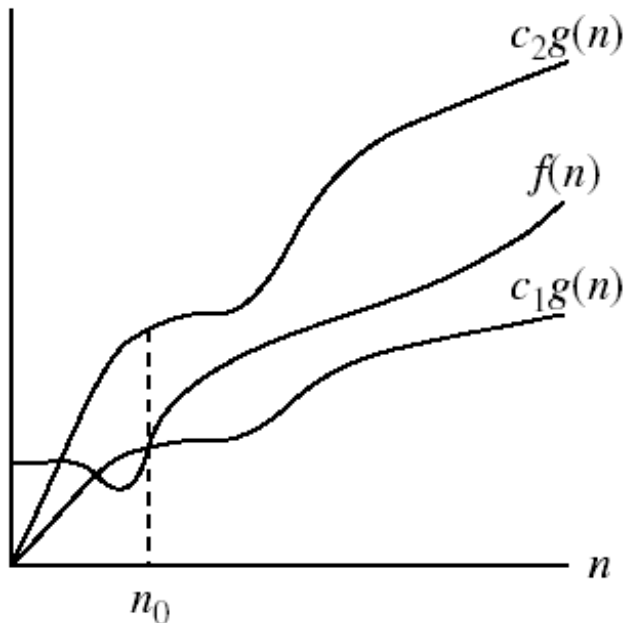
– $n = \Omega(2n), n^3 = \Omega(n^2), n = \Omega(\log n)$



Asymptotic notations (cont.)

- Θ -notation

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .$



- Intuitively $\Theta(g(n))$ = the set of functions with the same order of growth as $g(n)$

$g(n)$ is an *asymptotically tight bound* for $f(n)$.



Examples

- $n^2/2 - n/2 = \Theta(n^2)$

- $\frac{1}{2} n^2 - \frac{1}{2} n \leq \frac{1}{2} n^2 \quad \forall n \geq 0 \Rightarrow c_2 = \frac{1}{2}$

- $\frac{1}{2} n^2 - \frac{1}{2} n \geq \frac{1}{2} n^2 - \frac{1}{2} n * \frac{1}{2} n \quad (\forall n \geq 2) = \frac{1}{4} n^2 \Rightarrow c_1 = \frac{1}{4}$

- $n \neq \Theta(n^2): c_1 n^2 \leq n \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq 1/c_1$

- $6n^3 \neq \Theta(n^2): c_1 n^2 \leq 6n^3 \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq c_2 / 6$

- $n \neq \Theta(\log n): c_1 \log n \leq n \leq c_2 \log n$

$\Rightarrow c_2 \geq n/\log n, \forall n \geq n_0$ - impossible



Asymptotic Notations

- A way to describe behavior of functions in the limit
 - How we indicate running times of algorithms
 - Describe the running time of an algorithm as n grows to ∞
- O notation: asymptotic “less than”: $f(n) \leq g(n)$
- Ω notation: asymptotic “greater than”: $f(n) \geq g(n)$
- Θ notation: asymptotic “equality”: $f(n) = g(n)$



Comparisons of Functions

- *Theorem:*

$$f(n) = \Theta(g(n)) \Leftrightarrow f = O(g(n)) \text{ and } f = \Omega(g(n))$$

- **Transitivity:**

- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$
- Same for O and Ω

- **Reflexivity:**

- $f(n) = \Theta(f(n))$
- Same for O and Ω

- **Symmetry:**

- $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$

- **Transpose symmetry:**

- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$



Asymptotic Notations - Examples

- Θ notation

- $n^2/2 - n/2 = \Theta(n^2)$
- $(6n^3 + 1)\lg n / (n + 1) = \Theta(n^2 \lg n)$
- n vs. n^2 $n \neq \Theta(n^2)$

- Ω notation

- n vs. $2n$ $n = \Omega(2n)$
- n^3 vs. n^2 $n^3 = \Omega(n^2)$
- n vs. $\log n$ $n = \Omega(\log n)$
- n vs. n^2 $n \neq \Omega(n^2)$

- O notation

- $2n^2$ vs. n^3 $2n^2 = O(n^3)$
- n^2 vs. n^2 $n^2 = O(n^2)$
- n^3 vs. $n \log n$ $n^3 \neq O(n \lg n)$



Transitivity:

$$f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \text{ imply } f(n) = \Theta(h(n)) ,$$

$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \text{ imply } f(n) = O(h(n)) ,$$

$$f(n) = \Omega(g(n)) \text{ and } g(n) = \Omega(h(n)) \text{ imply } f(n) = \Omega(h(n)) ,$$

$$f(n) = o(g(n)) \text{ and } g(n) = o(h(n)) \text{ imply } f(n) = o(h(n)) ,$$

$$f(n) = \omega(g(n)) \text{ and } g(n) = \omega(h(n)) \text{ imply } f(n) = \omega(h(n)) .$$

Reflexivity:

$$f(n) = \Theta(f(n)) ,$$

$$f(n) = O(f(n)) ,$$

$$f(n) = \Omega(f(n)) .$$

Symmetry:

$$f(n) = \Theta(g(n)) \text{ if and only if } g(n) = \Theta(f(n)) .$$

Transpose symmetry:

$$f(n) = O(g(n)) \text{ if and only if } g(n) = \Omega(f(n)) ,$$

$$f(n) = o(g(n)) \text{ if and only if } g(n) = \omega(f(n)) .$$



$$\begin{array}{lll}
f(n) = O(g(n)) & \text{is like} & a \leq b , \\
f(n) = \Omega(g(n)) & \text{is like} & a \geq b , \\
f(n) = \Theta(g(n)) & \text{is like} & a = b , \\
f(n) = o(g(n)) & \text{is like} & a < b , \\
f(n) = \omega(g(n)) & \text{is like} & a > b .
\end{array}$$

Floors and ceilings

For any real number x , we denote the greatest integer less than or equal to x by $\lfloor x \rfloor$ (read “the floor of x ”) and the least integer greater than or equal to x by $\lceil x \rceil$ (read “the ceiling of x ”). For all real x ,

$$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1 . \quad (3.3)$$

Polynomials

Given a nonnegative integer d , a *polynomial in n of degree d* is a function $p(n)$ of the form

$$p(n) = \sum_{i=0}^d a_i n^i ,$$



Exponentials

For all real $a > 0$, m , and n , we have the following identities:

$$\begin{aligned}a^0 &= 1 , \\a^1 &= a , \\a^{-1} &= 1/a , \\(a^m)^n &= a^{mn} , \\(a^m)^n &= (a^n)^m , \\a^m a^n &= a^{m+n} .\end{aligned}$$

Logarithms

We shall use the following notations:

$$\begin{aligned}\lg n &= \log_2 n \quad (\text{binary logarithm}) \\ \ln n &= \log_e n \quad (\text{natural logarithm}) , \\ \lg^k n &= (\lg n)^k \quad (\text{exponentiation}) , \\ \lg \lg n &= \lg(\lg n) \quad (\text{composition}) .\end{aligned}$$

For all real $a > 0$, $b > 0$, $c > 0$, and n ,

$$\begin{aligned}a &= b^{\log_b a} , \\ \log_c(ab) &= \log_c a + \log_c b , \\ \log_b a^n &= n \log_b a , \\ \log_b a &= \frac{\log_c a}{\log_c b} , \\ \log_b(1/a) &= -\log_b a , \\ \log_b a &= \frac{1}{\log_a b} , \\ a^{\log_b c} &= c^{\log_b a} ,\end{aligned}$$



Polynomial-Time



Brute force. For many non-trivial problems, there is a natural brute force search algorithm that checks every possible solution.

- Typically takes 2^N time or worse for inputs of size N .
- Unacceptable in practice.

↖ $n!$ for stable matching
with n men and n women

Desirable scaling property. When the input size doubles, the algorithm should only slow down by some constant factor C .

There exists constants $c > 0$ and $d > 0$ such that on every input of size N , its running time is bounded by $c N^d$ steps.

Def. An algorithm is **poly-time** if the above scaling property holds.

↖ choose $C = 2^d$



Worst-Case Polynomial-Time



Def. An algorithm is **efficient** if its running time is polynomial.

Justification: It really works in practice!

- Although $6.02 \times 10^{23} \times N^{20}$ is technically poly-time, it would be useless in practice.
- In practice, the poly-time algorithms that people develop almost always have low constants and low exponents.
- Breaking through the exponential barrier of brute force typically exposes some crucial structure of the problem.

Exceptions.

- Some poly-time algorithms do have high constants and/or exponents, and are useless in practice.
- Some exponential-time (or worse) algorithms are widely used because the worst-case instances seem to be rare.

↖
simplex method
Unix grep



Sorting problem

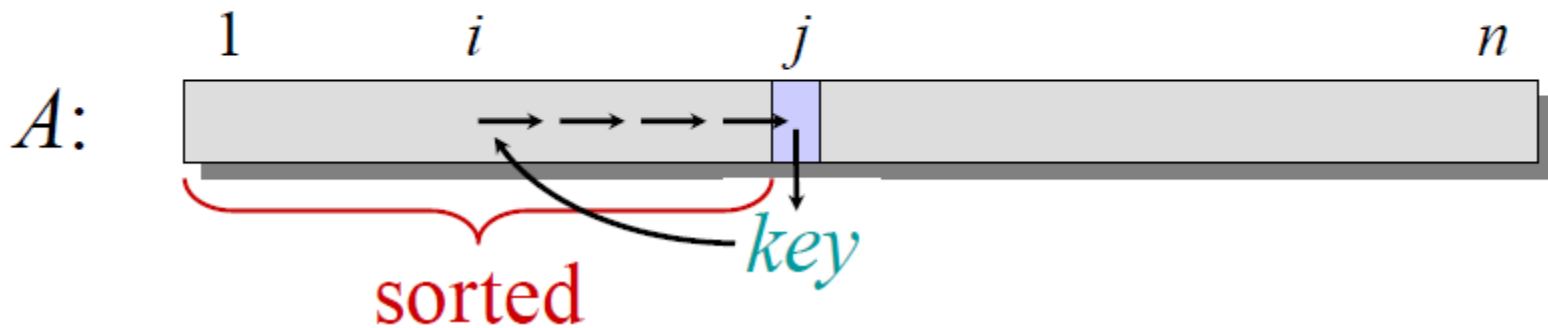
The sorting problem

- **Input:** a sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- **Output:** a permutation $\langle a_1', a_2', \dots, a_n' \rangle$, s.t. $a_1' \leq a_2' \leq \dots \leq a_n'$
- An **instance** of a problem:
 - the input needed to compute a solution (e.g.: $\langle 5, 3, 6, 2 \rangle$)
- An algorithm is **correct** if it ends with the correct output in a finite amount of time, on any legitimate input



“pseudocode”

```
INSERTION-SORT ( $A, n$ )    ▷  $A[1 \dots n]$   
  for  $j \leftarrow 2$  to  $n$   
    do  $key \leftarrow A[j]$   
       $i \leftarrow j - 1$   
      while  $i > 0$  and  $A[i] > key$   
        do  $A[i+1] \leftarrow A[i]$   
           $i \leftarrow i - 1$   
       $A[i+1] = key$ 
```



Correctness Analysis

- Loop invariant:

At the start of each iteration of the **for** loop of lines 1–8, the subarray $A[1 \dots j - 1]$ consists of the elements originally in $A[1 \dots j - 1]$, but in sorted order.

- **Initialization:** It is true prior to the first iteration of the loop.
- **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.



Insertion Sort

Statement	Effort
InsertionSort(A, n) {	
for i = 2 to n {	$c_1 n$
key = A[i]	$c_2(n-1)$
j = i - 1;	$c_3(n-1)$
while (j > 0) and (A[j] > key) {	$c_4 T$
A[j+1] = A[j]	$c_5(T-(n-1))$
j = j - 1	$c_6(T-(n-1))$
}	0
A[j+1] = key	$c_7(n-1)$
}	0
}	

$T = t_2 + t_3 + \dots + t_n$ where t_i is number of while expression evaluations for the i^{th} for loop iteration



Analyzing Insertion Sort

- $$T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4T + c_5(T - (n-1)) + c_6(T - (n-1)) + c_7(n-1)$$
$$= c_8T + c_9n + c_{10}$$
- What can T be?
 - Best case -- inner loop body never executed
 - ✓ $t_i = 1 \rightarrow T(n)$ is a linear function
 - Worst case -- inner loop body executed for all previous elements
 - ✓ $t_i = i \rightarrow T(n)$ is a quadratic function
 - Average case
 - ✓ ???



Analysis

- Simplifications
 - Ignore actual and abstract statement costs
 - *Order of growth* is the interesting measure:
 - Highest-order term is what counts
 - Remember, we are doing asymptotic analysis
 - As the input size grows larger it is the high order term that dominates



Upper Bound Notation

- We say InsertionSort's run time is $O(n^2)$
 - Properly we should say run time is *in* $O(n^2)$
 - Read O as “Big- O ” (you’ll also hear it as “order”)
- In general a function
 - $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- Formally
 - $O(g(n)) = \{ f(n): \exists \text{ positive constants } c \text{ and } n_0 \text{ such that } f(n) \leq c \cdot g(n) \forall n \geq n_0 \}$



Insertion Sort Is $O(n^2)$

- Proof

- Suppose runtime is $an^2 + bn + c$
 - If any of a , b , and c are less than 0 replace the constant with its absolute value
- $an^2 + bn + c \leq (a + b + c)n^2 + (a + b + c)n + (a + b + c)$
- $\leq 3(a + b + c)n^2$ for $n \geq 1$
- Let $c' = 3(a + b + c)$ and let $n_0 = 1$

- Question

- Is InsertionSort $O(n^3)$?
- Is InsertionSort $O(n)$?



- Merge Sort
- Master Theorem
- Divide-and conquer paradigm
- Other algorithms by D&C



Merge Sort

MERGE-SORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
```

Merge() takes two sorted subarrays of A and merges them into a single sorted subarray of A (how long should this take?)



MERGE(A, p, q, r)

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```



Analysis of Merge Sort

Statement	Effort
MergeSort(A, left, right) {	$T(n)$
if (left < right) {	$\Theta(1)$
mid = floor((left + right) / 2);	$\Theta(1)$
MergeSort(A, left, mid);	$T(n/2)$
MergeSort(A, mid+1, right);	$T(n/2)$
Merge(A, left, mid, right);	$\Theta(n)$
}	
}	

So $T(n) = \begin{matrix} \Theta(1) & \text{when } n = 1, \text{ and} \\ 2T(n/2) + \Theta(n) & \text{when } n > 1 \end{matrix}$

So what (more succinctly) is $T(n)$?



Recurrences

- The expression:

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

is a *recurrence*.

- **Recurrence**: an equation that describes a function in terms of its value on smaller functions

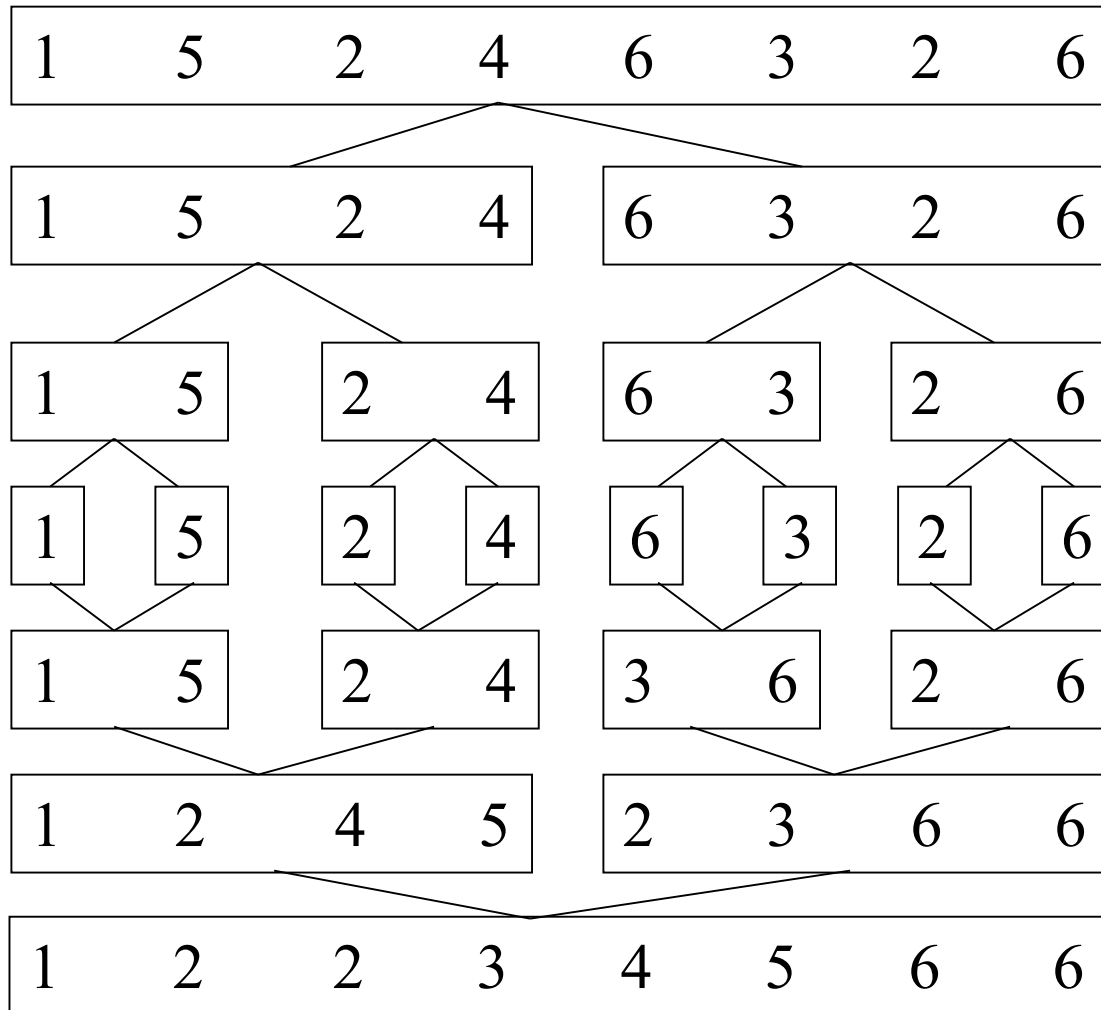


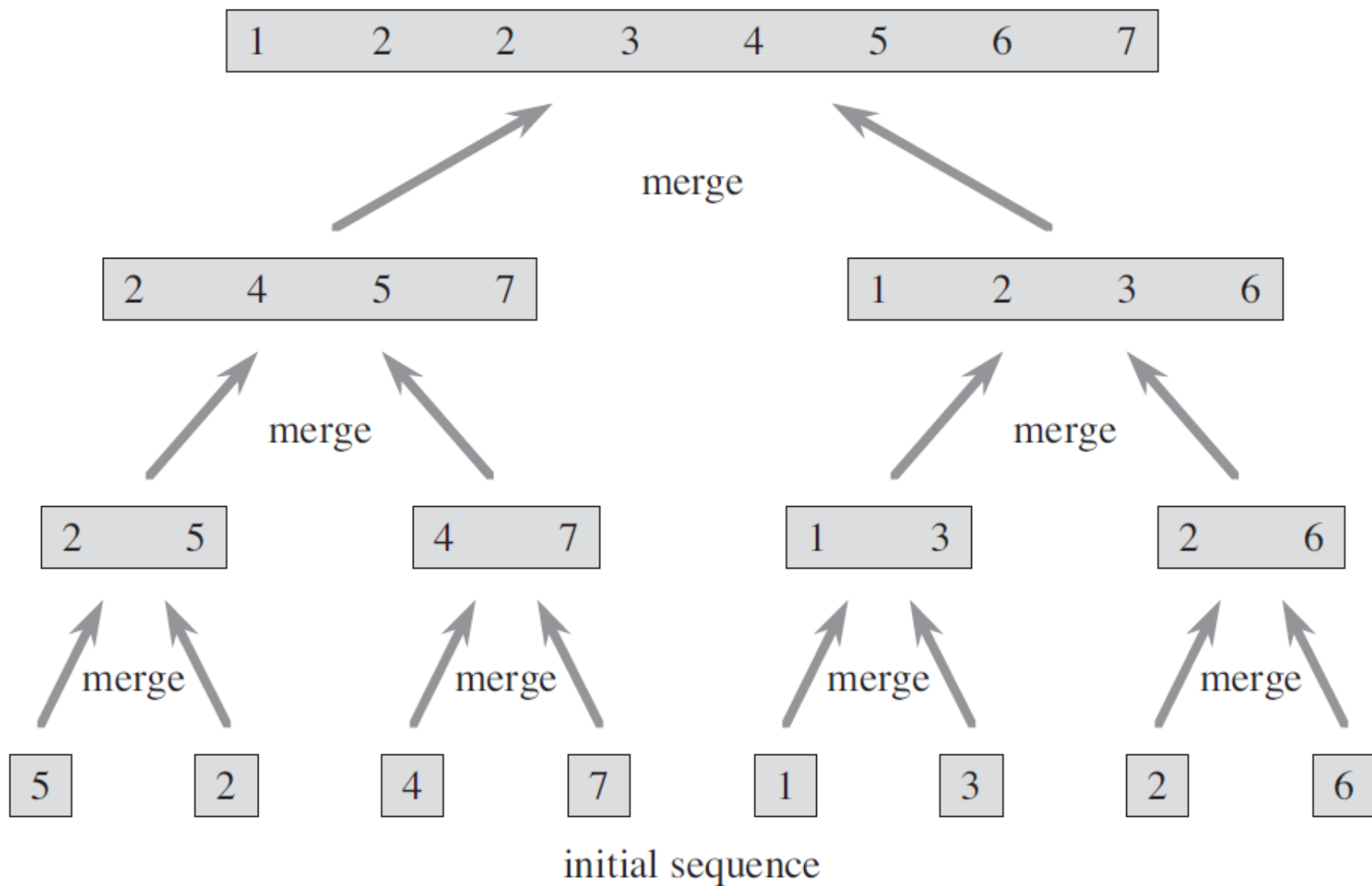
Structure of merge sort algorithm

1. **break** problem into similar (smaller) subproblems
2. **recursively solve** subproblems
3. **combine** solutions to produce final answer



Example of Merge Sort





Divide-and-conquer paradigm

1. *Divide* problem into subproblems.
2. *Conquer* subproblems by solving recursively.
3. *Combine* subproblem solutions.



Example of D&C Paradigm

Merge sort as Divide-and-conquer algorithm

1. Divide: Divide n -array into two $n/2$ -subarrays.
2. Conquer: Sort the two subarrays recursively.
3. Combine: Linear-time merge.



Recurrence for Merge sort

$$T(n) = \underset{\text{\#subproblems}}{2} \underset{\text{subproblem size}}{T(n/2)} + \underset{\text{work dividing \& combining}}{\Theta(n)}$$

$$T(n) = 2T(n/2) + \Theta(n)$$



A Useful Recurrence Relation

- Mergesort recurrence.

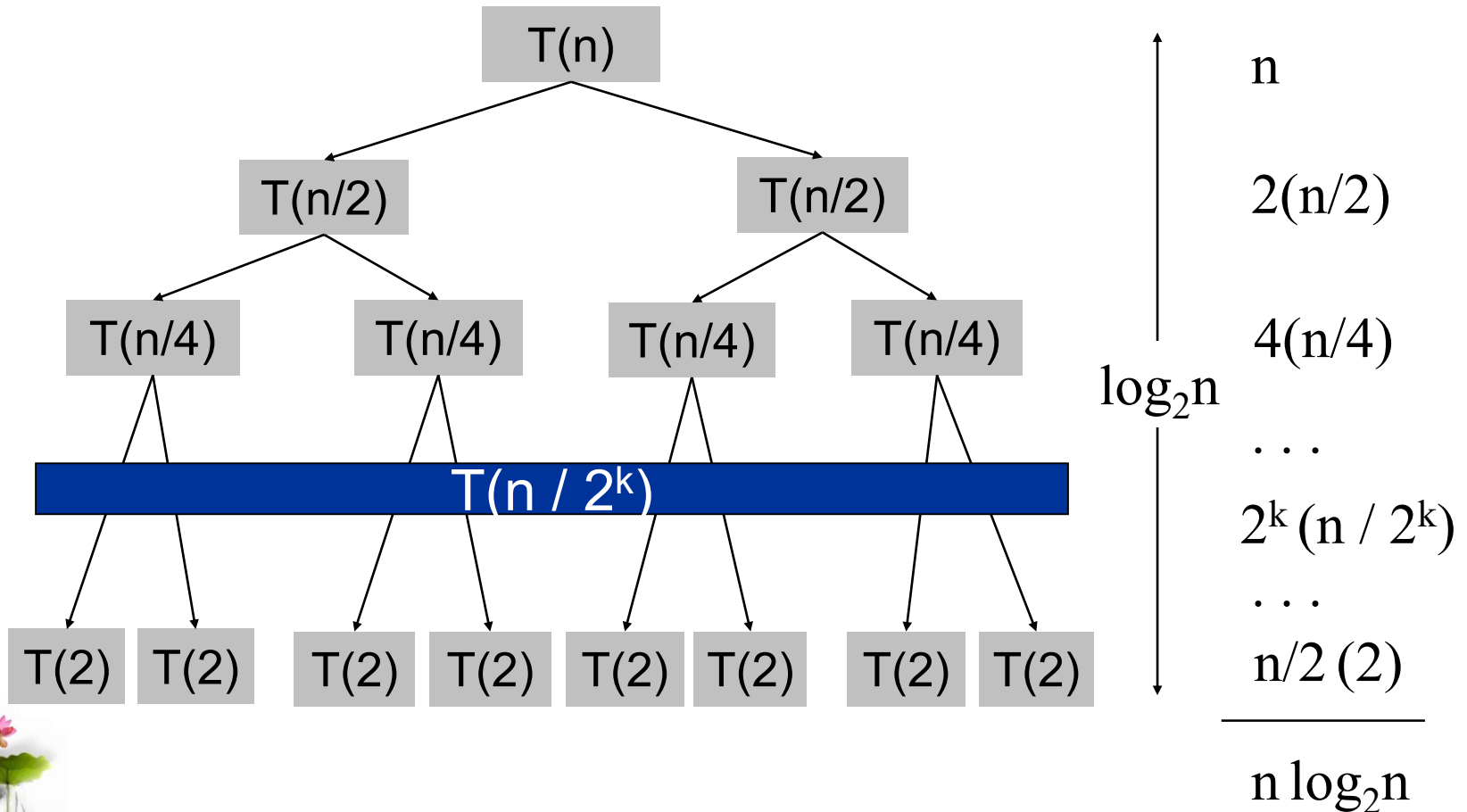
$$T(n) \leq \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

- Solution. $T(n) = O(n \log_2 n)$.
- Assorted proofs. We describe several ways to prove this recurrence. Initially we assume n is a power of 2 and replace $\leq$ with $=$.



Proof by Recursion Tree

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$



Proof by Telescoping

- Claim. If $T(n)$ satisfies this recurrence, then $T(n) = n \log_2 n$.

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

↑
assumes n is a power of 2

- Pf. For $n > 1$:

$$\begin{aligned} \frac{T(n)}{n} &= \frac{2T(n/2)}{n} + 1 \\ &= \frac{T(n/2)}{n/2} + 1 \\ &= \frac{T(n/4)}{n/4} + 1 + 1 \\ &\dots \\ &= \frac{T(n/n)}{n/n} + \underbrace{1 + \dots + 1}_{\log_2 n} \\ &= \log_2 n \end{aligned}$$



Proof by Induction

- Claim. If $T(n)$ satisfies this recurrence, then $T(n) = n \log_2 n$.

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

↑
assumes n is a power of 2

- Pf. (by induction on n)
 - Base case: $n = 1$.
 - Inductive hypothesis: $T(n) = n \log_2 n$.
 - Goal: show that $T(2n) = 2n \log_2 (2n)$.

$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$



Proof by Induction

- Claim. If $T(n)$ satisfies this recurrence, then $T(n) = n \lceil \lg n \rceil$.

$$T(n) \leq \begin{cases} 0 & \text{if } n=1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

↑
assumes n is a power of 2

- Pf. (by induction on n)

- Base case: $n = 1$.
- Define $n_1 = \lfloor n / 2 \rfloor$, $n_2 = \lceil n / 2 \rceil$.
- Induction step: assume true for $1, 2, \dots, n-1$.

$$\begin{aligned} T(n) &\leq T(n_1) + T(n_2) + n \\ &\leq n_1 \lceil \lg n_1 \rceil + n_2 \lceil \lg n_2 \rceil + n \\ &\leq n_1 \lceil \lg n_2 \rceil + n_2 \lceil \lg n_2 \rceil + n \\ &= n \lceil \lg n_2 \rceil + n \\ &\leq n(\lceil \lg n \rceil - 1) + n \\ &= n \lceil \lg n \rceil \end{aligned}$$

$$\begin{aligned} n_2 &= \lceil n/2 \rceil \\ &\leq \lceil 2^{\lceil \lg n \rceil} / 2 \rceil \\ &= 2^{\lceil \lg n \rceil} / 2 \\ &\Rightarrow \lg n_2 \leq \lceil \lg n \rceil - 1 \end{aligned}$$

Is there a Simple method?



Master method

- Idea: solve *class* of recurrences of form

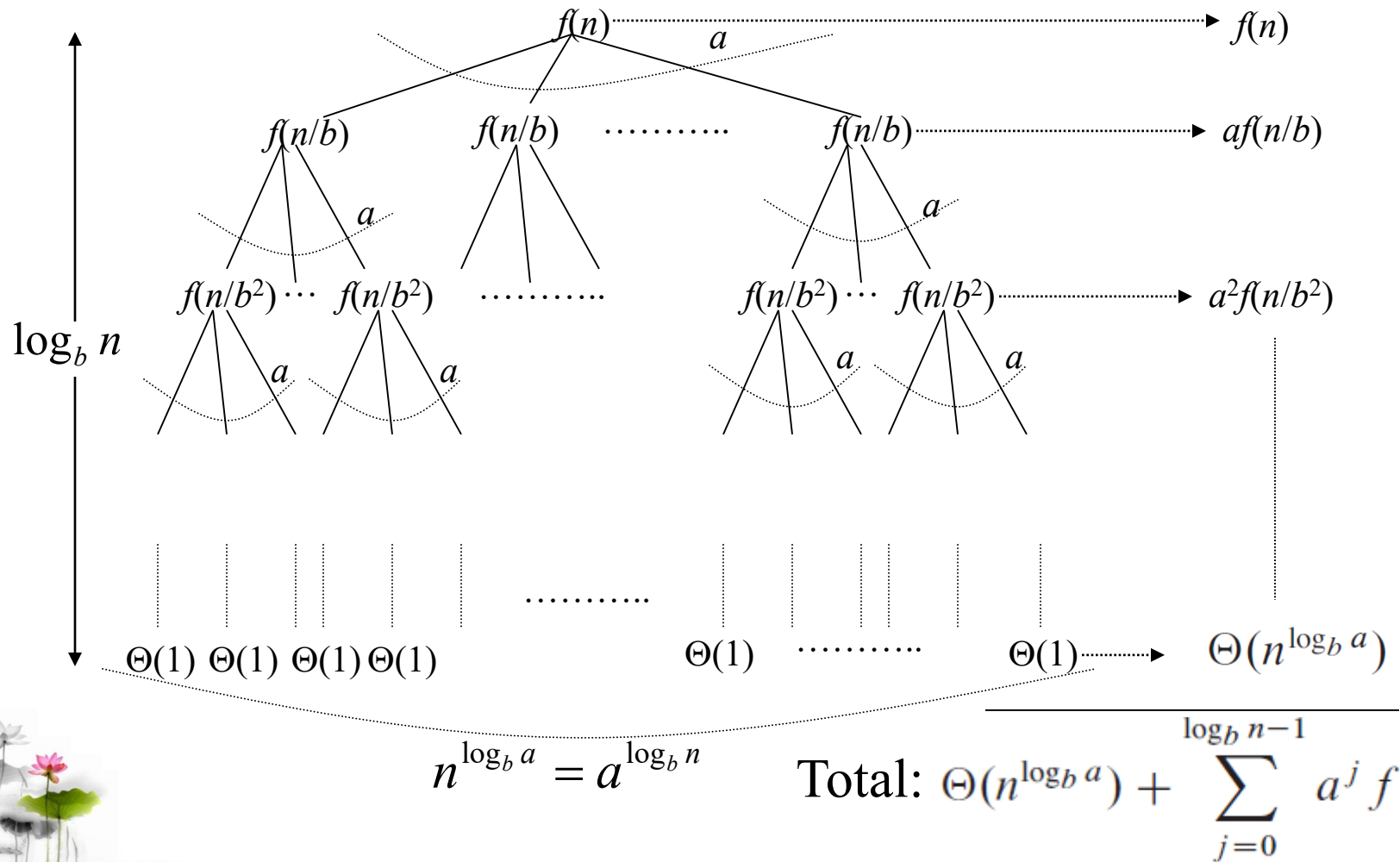
$$T(n) = aT(n/b) + f(n)$$

$a \geq 0$ and $b > 0$, and f asymptotically positive.

- Abstractly: $T(n)$ is runtime for an algorithm; it's unknown, but we do know that:
 - a subproblems of size n/b are solved recursively, each in time $T(n/b)$.
 - $f(n)$ is the cost of **dividing** problem (beforehand) and **combining** the results (afterward).



Recursion tree



Analysis

- #of leaves = $n^{\log_b a} = a^{\log_b n}$
- Iterating the recurrence (expanding the tree) yields

$$\begin{aligned} T(n) &= f(n) + aT(n/b) \\ &= f(n) + af(n/b) + a^2T(n/b^2) \\ &= f(n) + af(n/b) + a^2f(n/b^2) + \dots \\ &\quad + a^{\log_b n - 1} f(n / b^{\log_b n - 1}) + a^{\log_b n} T(1) \end{aligned}$$



Intuition

- Three common cases:
 - Running time dominated by cost at leaves.
 - Running time evenly distributed throughout tree
 - Running time dominated by cost at root
- Thus, to solve recurrence we need only characterize the **dominant** term in each case!



Master Theorem, Summarized

Master Theorem: $T(n) = aT(n/b) + f(n)$

1. $f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow T(n) = O(n^{\log_b a})$

2. $f(n) = \Theta(n^{\log_b a} \lg^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and $af(n/b) \leq cf(n)$
 $\Rightarrow T(n) = \Theta(f(n))$



MT Strategy

Basic strategy:

1. Extract a , b , and $f(n)$ from given recurrence
2. Determine $n^{\log_b a}$
3. Compare $f(n)$ and $n^{\log_b a}$ asymptotically
4. Determine appropriate MT case (if any), and apply it.



MT examples (cont.)

$$T(n) = 7T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} = n^{\lg 7}$$

$$f(n) / n^{\log_b a} = \Theta(n^2) / n^{\lg 7} = \Theta(n^{2-\lg 7}) = O(n^{-0.8})$$

$\Rightarrow$ Case 1, applies

$$T(n) = \Theta(n^{\lg 7})$$



MT examples (cont.)

- Binary **search** (not sort!):

$$T(n) = T(n/2) + \Theta(1); (a=1, b=2)$$

$$n^{\lg 1} = n^0 = 1$$

$$\Theta(1) / 1 = \Theta(\lg^0 n) = 1$$

$\Rightarrow$ Case 2,

$$T(n) = \Theta(\lg n)$$



MT examples (cont.)

$$T(n) = 4T(n/2) + n^3; (a=4, b=2)$$

$$n^3 / n^{\lg 4} = n \quad \Rightarrow \text{Case 3}$$

$$T(n) = \Theta(n^3)$$

Are we done?

No... Need to check regularity condition:

$$4f(n/2) \leq cf(n)$$

$$4n^3/8 \leq cn^3$$

$$n^3/2 \leq cn^3$$

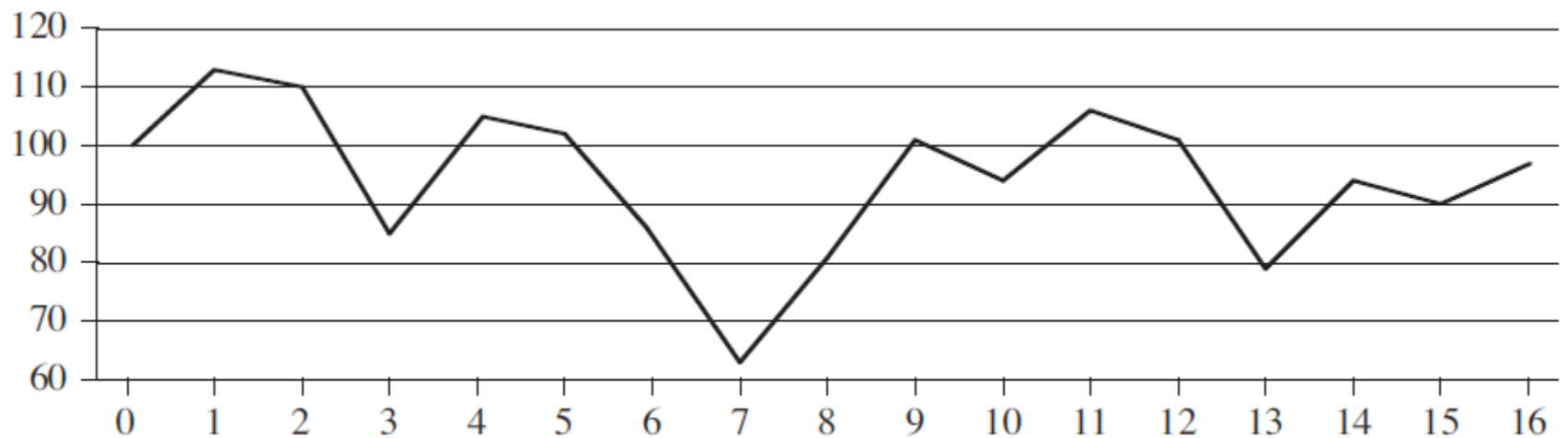
$$c=3/4 < 1$$



The maximum-subarray problem

- You are allowed to buy one unit of stock only one time and then sell it at a later date, buying and selling after the close of trading for the day. To compensate for this restriction, you are allowed to learn what the price of the stock will be in the future. Your goal is to maximize your profit.



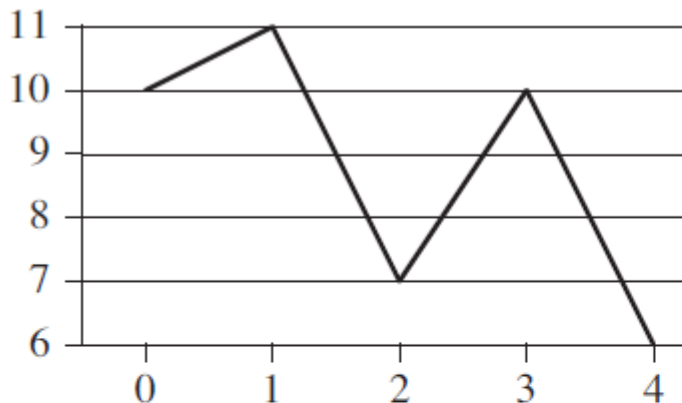


Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Figure 4.1 Information about the price of stock in the Volatile Chemical Corporation after the close of trading over a period of 17 days. The horizontal axis of the chart indicates the day, and the vertical axis shows the price. The bottom row of the table gives the change in price from the previous day.



Naïve idea: By buying at the lowest price or selling at the highest price.



Day	0	1	2	3	4
Price	10	11	7	10	6
Change		1	-4	3	-4

Figure 4.2 An example showing that the maximum profit does not always start at the lowest price or end at the highest price. Again, the horizontal axis indicates the day, and the vertical axis shows the price. Here, the maximum profit of \$3 per share would be earned by buying after day 2 and selling after day 3. The price of \$7 after day 2 is not the lowest price overall, and the price of \$10 after day 3 is not the highest price overall.



Transformation

- Change Idea: We want to find a sequence of days over which the net change from the first day to the last is maximum.
- The change on day i is the difference between the prices after day $i-1$ and after day i .
- $\rightarrow$ *Maximum Subarray Problem*



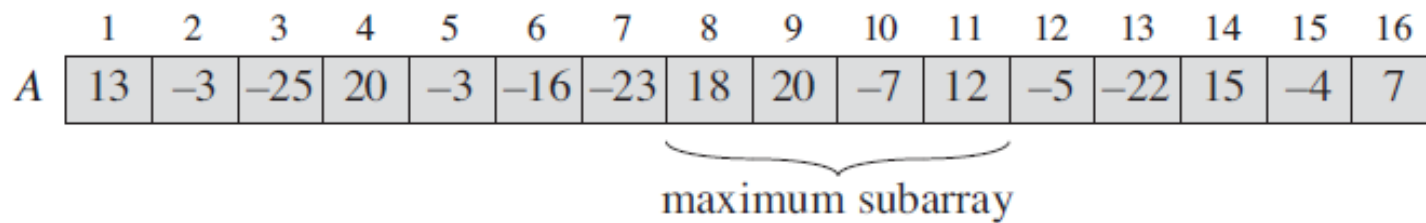


Figure 4.3 The change in stock prices as a maximum-subarray problem. Here, the subarray $A[8..11]$, with sum 43, has the greatest sum of any contiguous subarray of array A .



How to divide-and-conquer?

Target: to find a maximum subarray of $A[\text{low}..\text{high}]$.

Divide: We divide the subarray into two subarrays of as equal size as possible. So we find the midpoint mid and
 $A[\text{low}..\text{high}] \rightarrow A[\text{low}..\text{mid}] + A[\text{mid}..\text{high}]$.

Conquer:

Combine: $\max(A[\text{low}..\text{high}])$ must be either:

- entirely in the subarray $A[\text{low}..\text{mid}]$, so that $\text{low} \leq i \leq j \leq \text{mid}$,
- entirely in the subarray $A[\text{mid} + 1..\text{high}]$, so that $\text{mid} < i \leq j \leq \text{high}$, or
- crossing the midpoint, so that $\text{low} \leq i \leq \text{mid} < j \leq \text{high}$.

To find a maximum subarray that crosses the midpoint



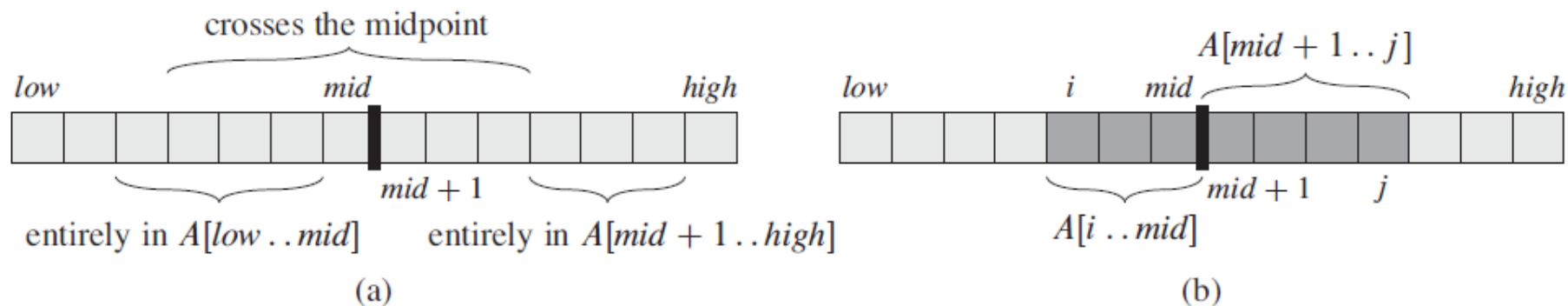


Figure 4.4 (a) Possible locations of subarrays of $A[low..high]$: entirely in $A[low..mid]$, entirely in $A[mid+1..high]$, or crossing the midpoint mid . (b) Any subarray of $A[low..high]$ crossing the midpoint comprises two subarrays $A[i..mid]$ and $A[mid+1..j]$, where $low \leq i \leq mid$ and $mid < j \leq high$.

Algorithm for the maximum-subarray problem

FIND-MAXIMUM-SUBARRAY($A, low, high$)

```
1  if  $high == low$ 
2      return ( $low, high, A[low]$ )           // base case: only one element
3  else  $mid = \lfloor (low + high) / 2 \rfloor$ 
4      ( $left-low, left-high, left-sum$ ) =
          FIND-MAXIMUM-SUBARRAY( $A, low, mid$ )
5      ( $right-low, right-high, right-sum$ ) =
          FIND-MAXIMUM-SUBARRAY( $A, mid + 1, high$ )
6      ( $cross-low, cross-high, cross-sum$ ) =
          FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )
7      if  $left-sum \geq right-sum$  and  $left-sum \geq cross-sum$ 
8          return ( $left-low, left-high, left-sum$ )
9      elseif  $right-sum \geq left-sum$  and  $right-sum \geq cross-sum$ 
10         return ( $right-low, right-high, right-sum$ )
11     else return ( $cross-low, cross-high, cross-sum$ )
```



Time complexity

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Master Theorem Case 2:

$$T(n) = \Theta(n \lg n)$$

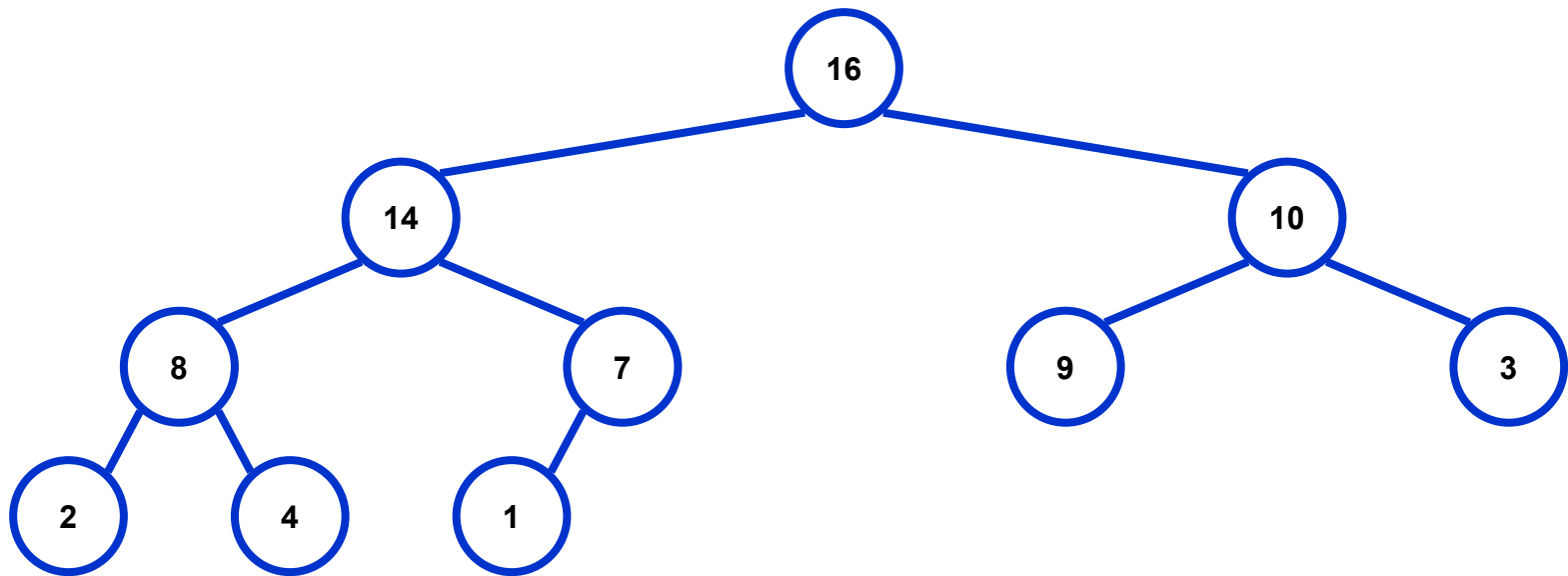


Heapsort



Heaps

□ A *heap* can be seen as a complete binary tree:



□ *What makes a binary tree complete?*

□ *Is the example above complete?*



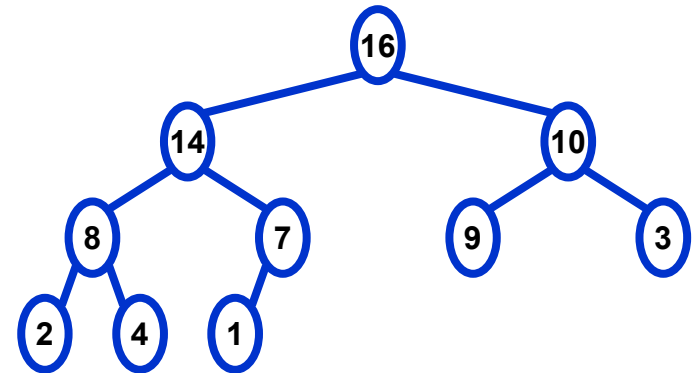
Heaps

- To represent a complete binary tree as an array:
 - The root node is $A[1]$
 - Node i is $A[i]$
 - The parent of node i is $A[i/2]$ (note: integer divide)
 - The left child of node i is $A[2i]$
 - The right child of node i is $A[2i + 1]$

$A =$

16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

 $=$



Referencing Heap Elements

□ So...

```
Parent(A[i]) { return A[⌊i/2⌋]; }
```

```
Left(A[i]) { return A[2*i]; }
```

```
right(A[i]) { return A[2*i + 1]; }
```



The Heap Property

□ Heaps also satisfy the *heap property*:

$$A[\textit{Parent}(A[i])] \geq A[i] \quad \text{for all nodes } i > 1$$

- In other words, the value of a node is at most the value of its parent
- *Where is the largest element in a heap stored?*



Heap Operations: Heapify()

- **Heapify()** : maintain the heap property
 - Given: a node i in the heap with children l and r
 - Given: two subtrees rooted at l and r , assumed to be heaps
 - Problem: The subtree rooted at i may violate the heap property (*How?*)
 - Action: let the value of the parent node “float down” so subtree at i satisfies the heap property
 - *What do you suppose will be the basic operation between i , l , and r ?*



Analyzing Heapify(): Formal

□ So we have

$$T(n) \leq T(2n/3) + \Theta(1)$$

□ By case 2 of the Master Theorem,

$$T(n) = O(\lg n)$$

□ Thus, **Heapify()** takes logarithmic time



Heap Operations: BuildHeap()

- We can build a heap in a bottom-up manner by running **Heapify()** on successive subarrays
 - Fact: for array of length n , all elements in range $A[\lfloor n/2 \rfloor + 1 .. n]$ are heaps (*Why?*)
 - So:
 - Walk backwards through the array from $n/2$ to 1, calling **Heapify()** on each node.
 - Order of processing guarantees that the children of node i are heaps when i is processed



BuildHeap()

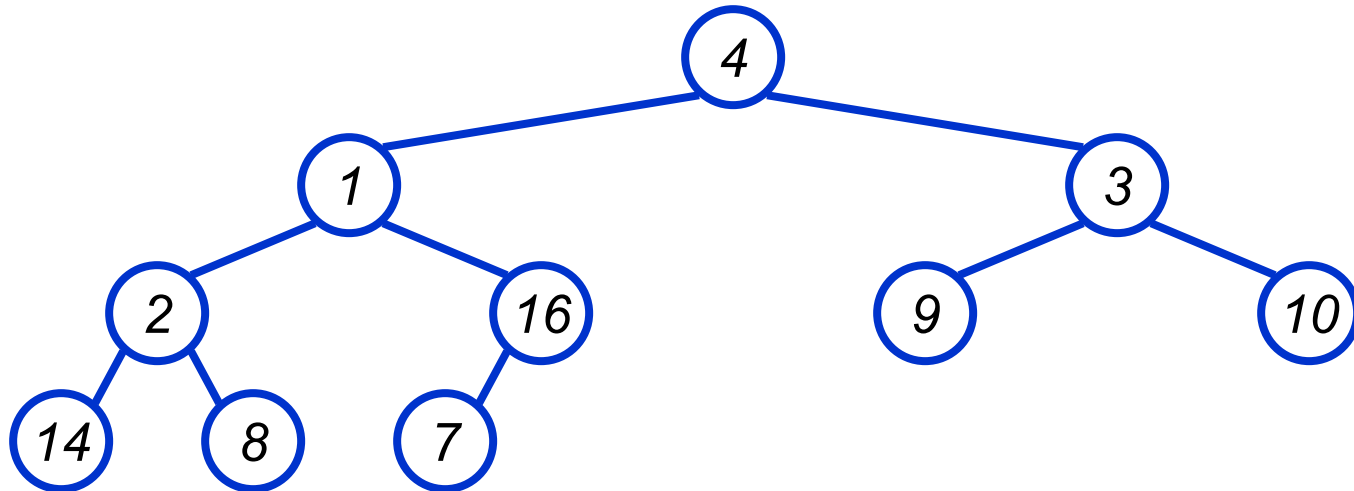
```
// given an unsorted array A, make A a heap
BuildHeap(A)
{
    heap_size(A) = length(A);
    for (i = ⌊length[A]/2⌋ downto 1)
        Heapify(A, i);
}
```



BuildHeap() Example

□ Work through example

$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$



Analyzing BuildHeap()

- Each call to **Heapify()** takes $O(\lg n)$ time
- There are $O(n)$ such calls (specifically, $\lfloor n/2 \rfloor$)
- Thus the running time is $O(n \lg n)$
 - *Is this a correct asymptotic upper bound?*
 - *Is this an asymptotically tight bound?*
- A tighter bound is $O(n)$
 - *How can this be? Is there a flaw in the above reasoning?*



Analyzing BuildHeap(): Tight

- To **Heapify** () a subtree takes $O(h)$ time where h is the height of the subtree
 - $h = O(\lg m)$, $m = \#$ nodes in subtree
 - The height of most subtrees is small
- Fact: an n -element heap has at most $\lceil n/2^{h+1} \rceil$ nodes of height h



Heapsort

- Given **BuildHeap()**, an in-place sorting algorithm is easily constructed:
 - Maximum element is at $A[1]$
 - Discard by swapping with element at $A[n]$
 - Decrement $\text{heap_size}[A]$
 - $A[n]$ now contains correct value
 - Restore heap property at $A[1]$ by calling **Heapify()**
 - Repeat, always swapping $A[1]$ for $A[\text{heap_size}(A)]$



Heapsort

Heapsort (A)

```
{  
    BuildHeap(A) ;  
    for (i = length(A) downto 2)  
    {  
        Swap(A[1], A[i]) ;  
        heap_size(A) -= 1 ;  
        Heapify(A, 1) ;  
    }  
}
```



Analyzing Heapsort

- The call to **BuildHeap** () takes $O(n)$ time
- Each of the $n - 1$ calls to **Heapify** () takes $O(\lg n)$ time
- Thus the total time taken by **HeapSort** ()
= $O(n) + (n - 1) O(\lg n)$
= $O(n) + O(n \lg n)$
= $O(n \lg n)$



Quicksort



Quicksort

- Sorts in place
- Sorts $O(n \lg n)$ in the average case
- Sorts $O(n^2)$ in the worst case
 - But in practice, it's quick
 - And the worst case doesn't happen often (but more on this later...)



Quicksort

- Another divide-and-conquer algorithm
 - The array $A[p..r]$ is *partitioned* into two non-empty subarrays $A[p..q]$ and $A[q+1..r]$
 - Invariant: All elements in $A[p..q]$ are less than all elements in $A[q+1..r]$
 - The subarrays are recursively sorted by calls to quicksort
 - Unlike merge sort, no combining step: two subarrays form an already-sorted array



Divide-and-conquer

1. *Divide*: Partition the array into two subarrays around a pivot x such that elements in lower subarray $\leq x \leq$ elements in upper subarray.



2. *Conquer*: Recursively sort the two subarrays.
3. *Combine*: Trivial (because in place).

Key: Linear-time partitioning procedure.



Quicksort Code

```
Quicksort(A, p, r)
{
    if (p < r)
    {
        q = Partition(A, p, r);
        Quicksort(A, p, q);
        Quicksort(A, q+1, r);
    }
}
```



Partition

- Clearly, all the action takes place in the **partition()** function
 - Rearranges the subarray in place
 - End result:
 - Two subarrays
 - All values in first subarray $\leq$ all values in second
 - Returns the index of the “pivot” element separating the two subarrays
- *How do you suppose we implement this?*



Partition In Words

- Partition(A, p, r):
 - Select an element to act as the “pivot” (*which?*)
 - Grow two regions, A[p..i] and A[j..r]
 - All elements in A[p..i] $\leq$ pivot
 - All elements in A[j..r] $\geq$ pivot
 - Increment i until A[i] $\geq$ pivot
 - Decrement j until A[j] $\leq$ pivot
 - Swap A[i] and A[j]
 - Repeat until i $\geq$ j
 - Return j

*Note: slightly different from
book's partition()*

Partition Code

```
Partition(A, p, r)
    x = A[p];
    i = p - 1;
    j = r + 1;
    while (TRUE)
        repeat
            j--;
        until A[j] <= x;
        repeat
            i++;
        until A[i] >= x;
        if (i < j)
            Swap(A, i, j);
        else
            return j;
```

Illustrate on

A = {5, 3, 2, 6, 4, 1, 3, 7};

*What is the running time of
partition()?*



Partition Code

```
Partition(A, p, r)
    x = A[p];
    i = p - 1;
    j = r + 1;
    while (TRUE)
        repeat
            j--;
        until A[j] <= x;
        repeat
            i++;
        until A[i] >= x;
        if (i < j)
            Swap(A, i, j);
        else
            return j;
```

partition() runs in $O(n)$ time



Analyzing Quicksort

- *What will be the worst case for the algorithm?*
 - Partition is always unbalanced
- *What will be the best case for the algorithm?*
 - Partition is perfectly balanced
- *Which is more likely?*
 - The latter, by far, except...
- *Will any particular input elicit the worst case?*
 - Yes: Already-sorted input



Analyzing Quicksort

- In the worst case:

$$T(1) = \Theta(1)$$

$$T(n) = T(n - 1) + \Theta(n)$$

- Works out to

$$T(n) = \Theta(n^2)$$



Analyzing Quicksort

- In the best case:

$$T(n) = 2T(n/2) + \Theta(n)$$

- What does this work out to?

$$T(n) = \Theta(n \lg n)$$



Quicksort in practice

- Quicksort is great general-purpose sorting algorithm.
- Quicksort is typically over twice as fast as Merge sort.
- Quicksort can benefit substantially from *code tuning*.
- Quicksort behaves well even with caching and virtual memory.



How fast can we sort?

All the sorting algorithms we have seen so far are *comparison sorts*: only use comparisons to determine the relative order of elements.

- *E.g.*, insertion sort, merge sort, quicksort, heapsort.

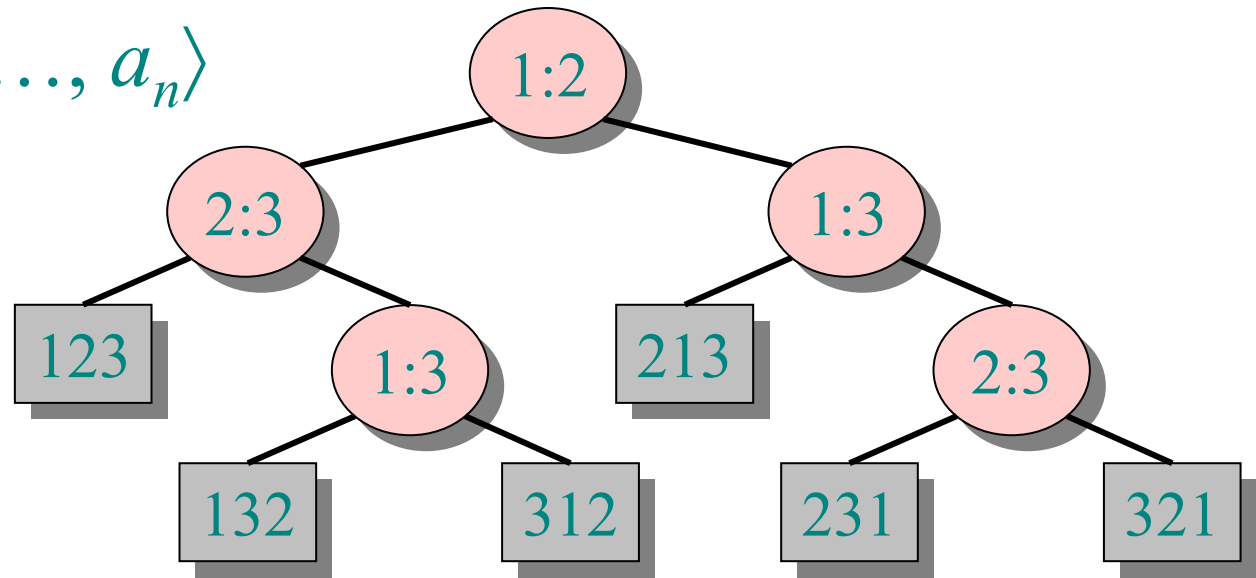
The best worst-case running time that we've seen for comparison sorting is $O(n \lg n)$.

Is $O(n \lg n)$ the best we can do?

Decision trees can help us answer this question.

Decision-tree example

Sort $\langle a_1, a_2, \dots, a_n \rangle$



Each internal node is labeled $i:j$ for $i, j \in \{1, 2, \dots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \leq a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

Lower bound for decision-tree sorting

Theorem. Any decision tree that can sort n elements must have height $\Omega(n \lg n)$.

Proof. The tree must contain $\geq n!$ leaves, since there are $n!$ possible permutations. A height- h binary tree has $\leq 2^h$ leaves. Thus, $n! \leq 2^h$.

$$\begin{aligned} \therefore h &\geq \lg(n!) && (\lg \text{ is mono. increasing}) \\ &\geq \lg((n/e)^n) && (\text{Stirling's formula}) \\ &= n \lg n - n \lg e \\ &= \Omega(n \lg n). \quad \square \end{aligned}$$

Sorting in linear time

Counting sort: No comparisons between elements.

- **Input:** $A[1 \dots n]$, where $A[j] \in \{1, 2, \dots, k\}$.
- **Output:** $B[1 \dots n]$, sorted.
- **Auxiliary storage:** $C[1 \dots k]$.

Analysis

$\Theta(k)$ { **for** $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$

$\Theta(n)$ { **for** $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$ { **for** $i \leftarrow 2$ **to** k
 do $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$ { **for** $j \leftarrow n$ **downto** 1
 do $B[C[A[j]]] \leftarrow A[j]$
 $C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$

Running time

If $k = O(n)$, then counting sort takes $\Theta(n)$ time.

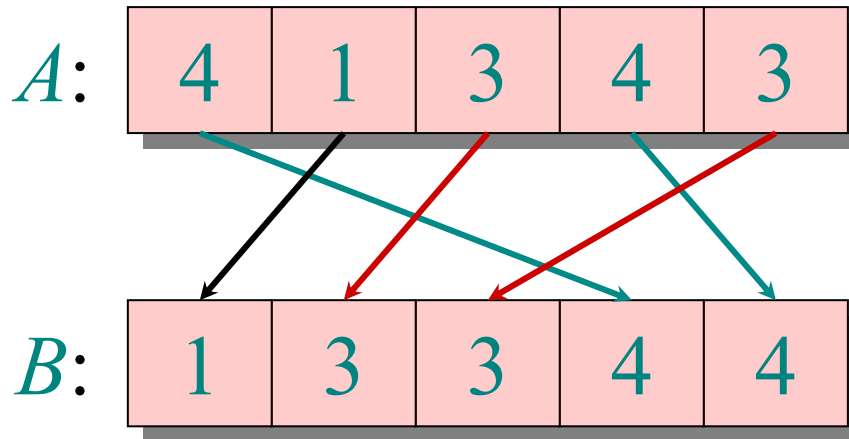
- But, sorting takes $\Omega(n \lg n)$ time!
- Where's the fallacy?

Answer:

- *Comparison sorting* takes $\Omega(n \lg n)$ time.
- Counting sort is not a *comparison sort*.
- In fact, not a single comparison between elements occurs!

Stable sorting

Counting sort is a *stable* sort: it preserves the input order among equal elements.

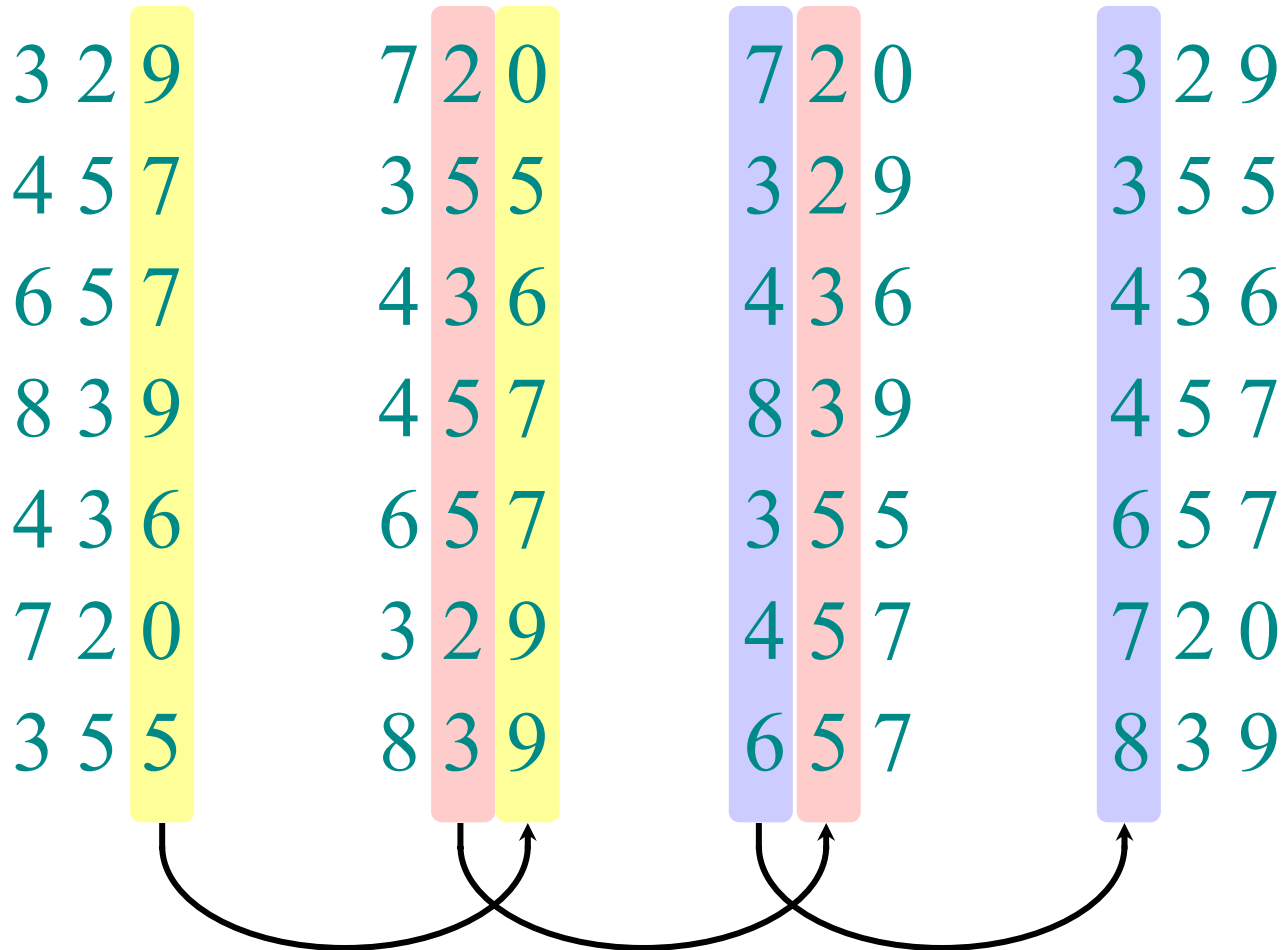


Exercise: What other sorts have this property?

Radix sort

- *Origin*: Herman Hollerith's card-sorting machine for the 1890 U.S. Census. (See Appendix  .)
- Digit-by-digit sort.
- Hollerith's original (bad) idea: sort on most-significant digit first.
- Good idea: Sort on *least-significant digit first* with auxiliary *stable* sort.

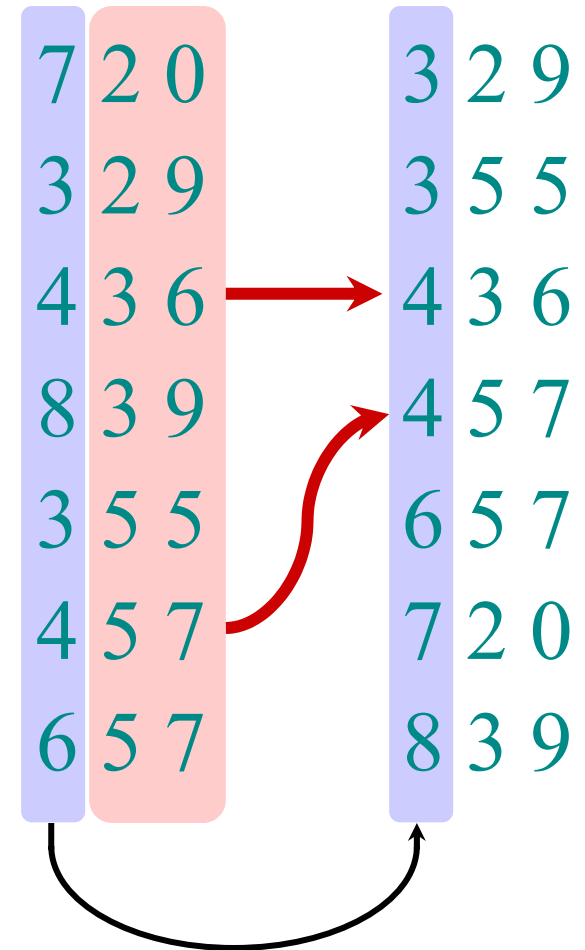
Operation of radix sort



Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
 - Two numbers that differ in digit t are correctly sorted.
 - Two numbers equal in digit t are put in the same order as the input $\Rightarrow$ correct order.



Order statistics

Select the i th smallest of n elements (the element with *rank* i).

- $i = 1$: *minimum*;
- $i = n$: *maximum*;
- $i = \lfloor (n+1)/2 \rfloor$ or $\lceil (n+1)/2 \rceil$: *median*.

Naive algorithm: Sort and index i th element.

Worst-case running time = $\Theta(n \lg n) + \Theta(1)$
= $\Theta(n \lg n)$,

using merge sort or heapsort (*not* quicksort).

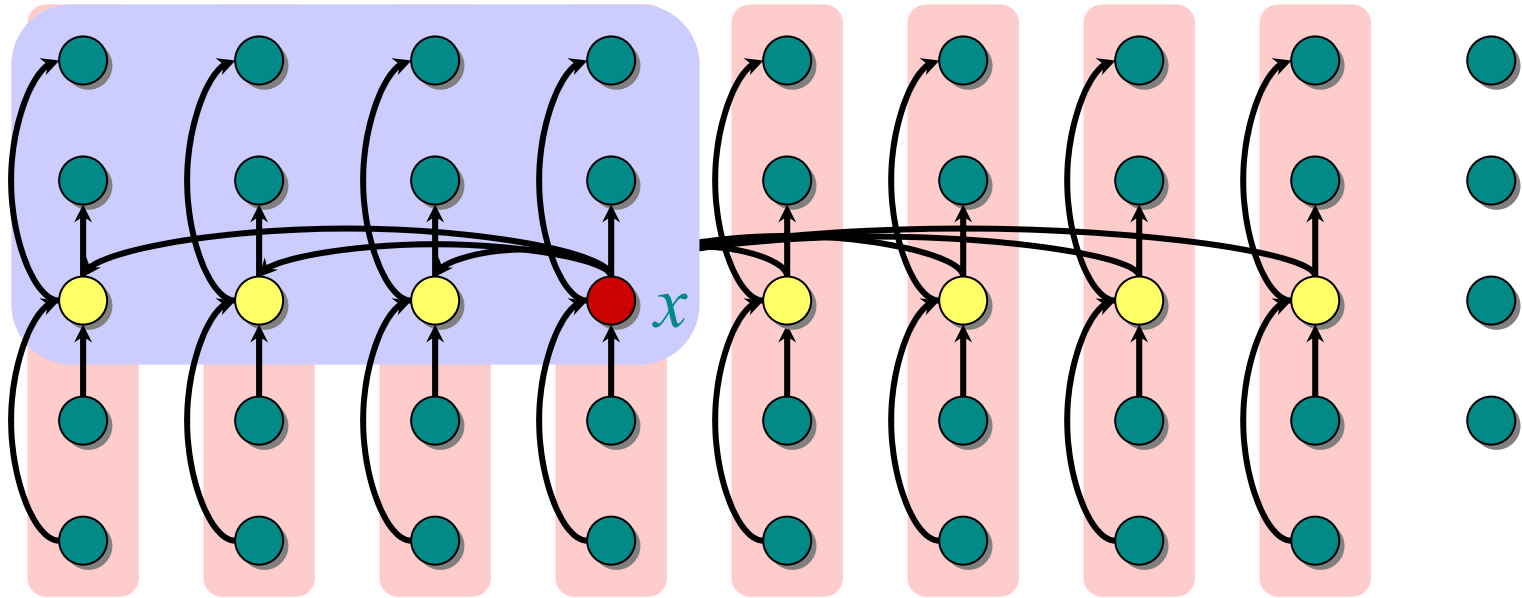
Worst-case linear-time order statistics

SELECT(i, n)

1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.
3. Partition around the pivot x . Let $k = \text{rank}(x)$.
4. **if** $i = k$ **then return** x
elseif $i < k$
 then recursively SELECT the i th
 smallest element in the lower part
else recursively SELECT the $(i-k)$ th
 smallest element in the upper part

Same as
RAND-
SELECT

Analysis (Assume all elements are distinct.)



At least half the group medians are $\leq x$, which is at least $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$ group medians.

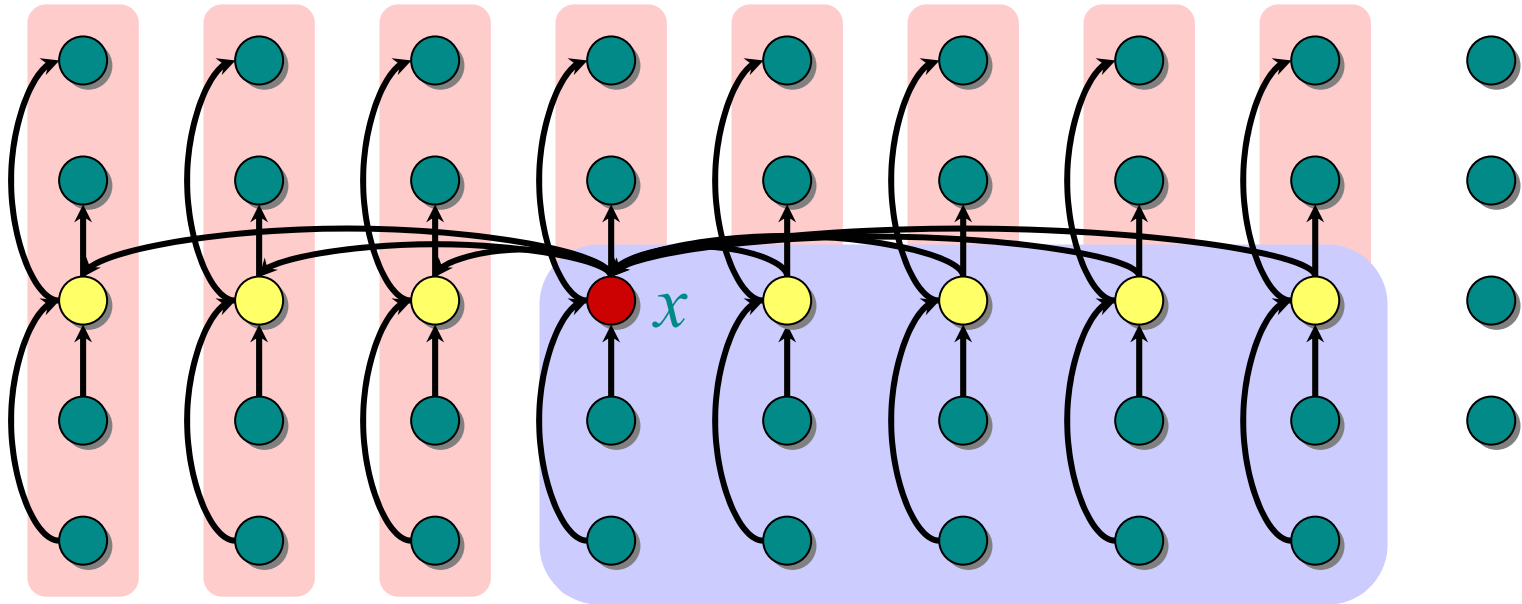
- Therefore, at least $3\lfloor n/10 \rfloor$ elements are $\leq x$.

lesser



greater

Analysis (Assume all elements are distinct.)



At least half the group medians are $\leq x$, which is at least $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$ group medians.

- Therefore, at least $3\lfloor n/10 \rfloor$ elements are $\leq x$.
- Similarly, at least $3\lfloor n/10 \rfloor$ elements are $\geq x$.

lesser



greater

Minor simplification

- For $n \geq 50$, we have $3\lfloor n/10 \rfloor \geq n/4$.
- Therefore, for $n \geq 50$ the recursive call to SELECT in Step 4 is executed recursively on $\leq 3n/4$ elements.
- Thus, the recurrence for running time can assume that Step 4 takes time $T(3n/4)$ in the worst case.
- For $n < 50$, we know that the worst-case time is $T(n) = \Theta(1)$.

Developing the recurrence

<u>$T(n)$</u>	SELECT(i, n)
$\Theta(n)$	1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
$T(n/5)$	2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.
$\Theta(n)$	3. Partition around the pivot x . Let $k = \text{rank}(x)$.
$T(3n/4)$	4. if $i = k$ then return x elseif $i < k$ then recursively SELECT the i th smallest element in the lower part else recursively SELECT the $(i-k)$ th smallest element in the upper part

Solving the recurrence

$$T(n) = T\left(\frac{1}{5}n\right) + T\left(\frac{3}{4}n\right) + \Theta(n)$$

Substitution:

$$T(n) \leq cn$$

$$T(n) \leq \frac{1}{5}cn + \frac{3}{4}cn + \Theta(n)$$

$$= \frac{19}{20}cn + \Theta(n)$$

$$= cn - \left(\frac{1}{20}cn - \Theta(n)\right)$$

$$\leq cn ,$$

if c is chosen large enough to handle both the $\Theta(n)$ and the initial conditions.

Dynamic Programming



Basic Steps

- 1. Characterize the structure of an optimal solution.
- 2. Recursively define the value of an optimal solution.
- 3. Compute the value of an optimal solution, typically in a bottom-up fashion.
- 4. Construct an optimal solution from computed information.



Rod cutting

The *rod-cutting problem* is the following. Given a rod of length n inches and a table of prices p_i for $i = 1, 2, \dots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces. Note that if the price p_n for a rod of length n is large enough, an optimal solution may require no cutting at all.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

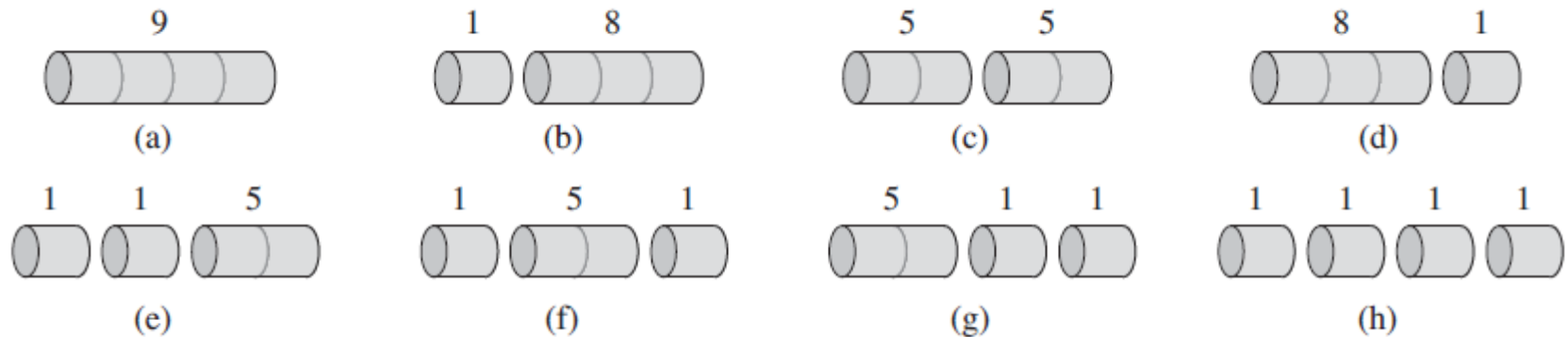


Figure 15.2 The 8 possible ways of cutting up a rod of length 4. Above each piece is the value of that piece, according to the sample price chart of Figure 15.1. The optimal strategy is part (c)—cutting the rod into two pieces of length 2—which has total value 10.



Rod cutting

an optimal decomposition

$$n = i_1 + i_2 + \cdots + i_k$$

provides maximum corresponding revenue

$$r_n = p_{i_1} + p_{i_2} + \cdots + p_{i_k} .$$

optimal substructure : recurrence function

$$r_n = \max (p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

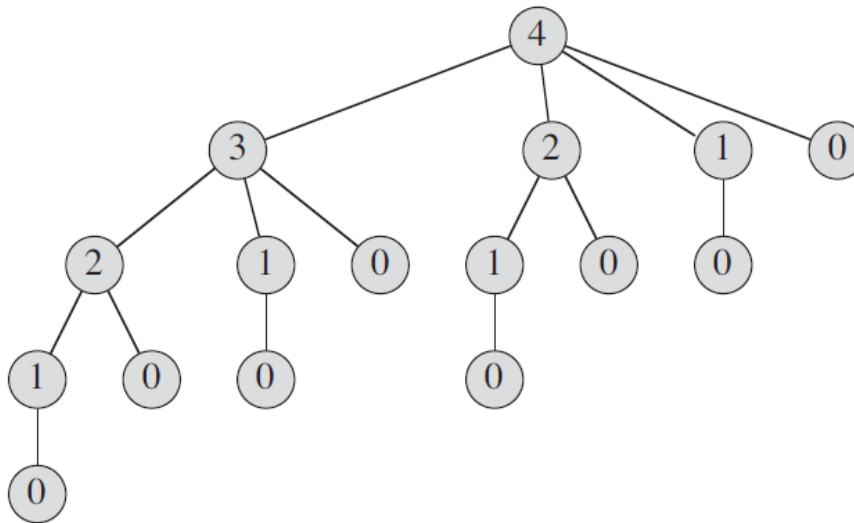
$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$



Recursive top-down implementation

CUT-ROD(p, n)

```
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$ 
6  return  $q$ 
```



$$T(n) = 1 + \sum_{j=0}^{n-1} T(j)$$

$$T(n) = 2^n$$



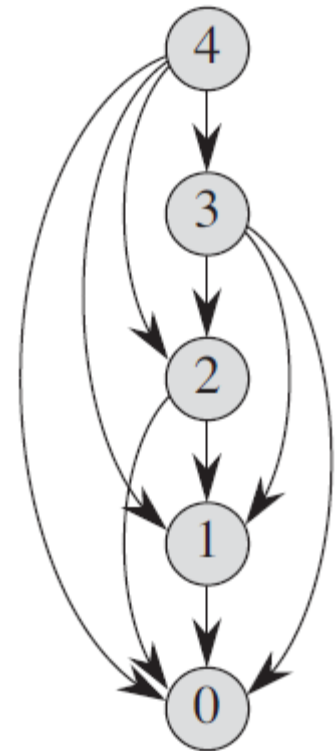
Top-down with memoization

MEMOIZED-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
2  for  $i = 0$  to  $n$ 
3       $r[i] = -\infty$ 
4  return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
```

MEMOIZED-CUT-ROD-AUX(p, n, r)

```
1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 
```



Bottom-up method

BOTTOM-UP-CUT-ROD(p, n)

```

1  let  $r[0..n]$  be a new array
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6           $q = \max(q, p[i] + r[j - i])$ 
7       $r[j] = q$ 
8  return  $r[n]$ 

```

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



Bottom-up method: Reconstructing a solution

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0..n]$  and  $s[0..n]$  be new arrays
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6          if  $q < p[i] + r[j - i]$ 
7               $q = p[i] + r[j - i]$ 
8               $s[j] = i$ 
9       $r[j] = q$ 
10 return  $r$  and  $s$ 
```

PRINT-CUT-ROD-SOLUTION(p, n)

```
1   $(r, s) = \text{EXTENDED-BOTTOM-UP-CUT-ROD}(p, n)$ 
2  while  $n > 0$ 
3      print  $s[n]$ 
4       $n = n - s[n]$ 
```



Knapsack problem

Given some items, pack the knapsack to get the maximum total value. Each item has some weight and some value. Total weight that we can carry is no more than some fixed number W . So we must consider weights of items as well as their value.

Item #	Weight	Value
1	1	8
2	3	6
3	5	5



Knapsack problem

There are two versions of the problem:

- (1) “0-1 knapsack problem” and
- (2) “Fractional knapsack problem”

- (1) Items are indivisible; you either take an item or not. Solved with *dynamic programming*
- (2) Items are divisible: you can take any fraction of an item. Solved with a *greedy algorithm*.



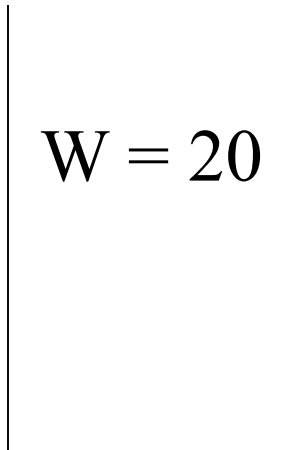
0-1 Knapsack problem

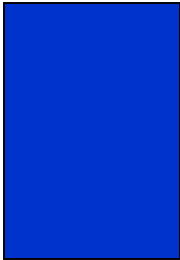
- Given a knapsack with maximum capacity W , and a set S consisting of n items
- Each item i has some weight w_i and benefit value b_i (all w_i , b_i and W are integer values)
- Problem: How to pack the knapsack to achieve maximum total value of packed items?



0-1 Knapsack problem: a picture

This is a knapsack
Max weight: $W = 20$



	Weight	Benefit value
Items	w_i	b_i
	2	3
	3	4
	4	5
	5	8
	9	10



Recursive Formula for subproblems

- Recursive formula for subproblems:

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max \{B[k-1, w], B[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- It means, that the best subset of S_k that has total weight w is one of the two:
 - 1) the best subset of S_{k-1} that has total weight w , **or**
 - 2) the best subset of S_{k-1} that has total weight $w - w_k$ plus the item k



Recursive Formula

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max \{B[k-1, w], B[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- The best subset of S_k that has the total weight w , either contains item k or not.
- First case: $w_k > w$. Item k can't be part of the solution, since if it was, the total weight would be $> w$, which is unacceptable
- Second case: $w_k \leq w$. Then the item k can be in the solution, and we choose the case with greater value



0-1 Knapsack Algorithm

for $w = 0$ to W

$$B[0,w] = 0$$

for $i = 0$ to n

$$B[i,0] = 0$$

for $w = 0$ to W

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$$B[i, w] = b_i + B[i-1, w - w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else $B[i, w] = B[i-1, w]$ // $w_i > w$



Running time

for $w = 0$ to W

$O(W)$

$B[0,w] = 0$

for $i = 0$ to n

Repeat n times

$B[i,0] = 0$

for $w = 0$ to W

$O(W)$

< the rest of the code >

What is the running time of this algorithm?

$O(n*W)$

Remember that the brute-force algorithm
takes $O(2^n)$



Example

Let's run our algorithm on the following data:

$n = 4$ (# of elements)

$W = 5$ (max weight)

Elements (weight, benefit):

(2,3), (3,4), (4,5), (5,6)



Example (17)

	i	0	1	2	3	4
W						
0		0	0	0	0	0
1		0	0	0	0	0
2		0	3	3	3	3
3		0	3	4	4	4
4		0	3	4	5	5
5		0	3	7	7	7

$i=3$
 $b_i=5$
 $w_i=4$
 $w=5$

Items:
1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$



Comments

- This algorithm only finds the max possible value that can be carried in the knapsack
- To know the items that make this maximum value, an addition to this algorithm is necessary
- Please see LCS algorithm from the previous lecture for the example how to extract this data from the table we built



Conclusion

- Dynamic programming is a useful technique of solving certain kind of problems
- When the solution can be recursively described in terms of partial solutions, we can store these partial solutions and re-use them as necessary
- Running time (Dynamic Programming algorithm vs. naïve algorithm):
 - LCS: $O(m * n)$ vs. $O(n * 2^m)$
 - 0-1 Knapsack problem: $O(W * n)$ vs. $O(2^n)$





Chapter 16

Greedy Algorithms



Greedy Algorithms

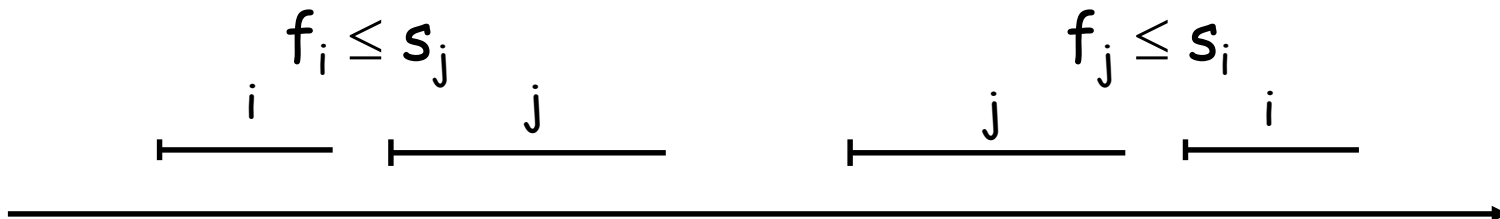
- Similar to dynamic programming, but simpler approach
 - Also used for optimization problems
- **Idea:** When we have a choice to make, make the one that looks best right now
 - Make a locally optimal choice in hope of getting a globally optimal solution
- Greedy algorithms don't always yield an optimal solution
- When the problem has certain general characteristics, greedy algorithms give optimal solutions

Activity Selection

- Schedule n **activities** that require exclusive use of a common resource

$S = \{a_1, \dots, a_n\}$ – set of activities

- a_i needs resource during period $[s_i, f_i)$
 - s_i = **start time** and f_i = **finish time** of activity a_i
 - $0 \leq s_i < f_i < \infty$
- Activities a_i and a_j are **compatible** if the intervals $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap



Activity Selection Problem

Select the largest possible set of nonoverlapping (*mutually compatible*) activities.

E.g.:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

- Activities are sorted in increasing order of finish times
- A subset of mutually compatible activities: $\{a_3, a_9, a_{11}\}$
- Maximal set of mutually compatible activities:
 $\{a_1, a_4, a_8, a_{11}\}$ and $\{a_2, a_4, a_9, a_{11}\}$

Why is the Theorem Useful?

	Dynamic programming	Using the theorem
Number of subproblems in the optimal solution	2 subproblems: S_{ik}, S_{kj}	1 subproblem: S_{mj} $S_{im} = \emptyset$
Number of choices to consider	$j - i - 1$ choices	1 choice: the activity with the earliest finish time in S_{ij}

- Making the greedy choice (the activity with the earliest finish time in S_{ij})
 - Reduce the number of subproblems and choices
 - Solve each subproblem in a top-down fashion

Greedy Approach

- To select a maximum size subset of mutually compatible activities from set S_{ij} :
 - Choose $a_m \in S_{ij}$ with earliest finish time (greedy choice)
 - Add a_m to the set of activities used in the optimal solution
 - Solve the same problem for the set S_{mj}
- From the theorem
 - By choosing a_m we are guaranteed to have used an activity included in an optimal solution
 - $\Rightarrow$ We do not need to solve the subproblem S_{mj} before making the choice!
 - The problem has the **GREEDY CHOICE** property

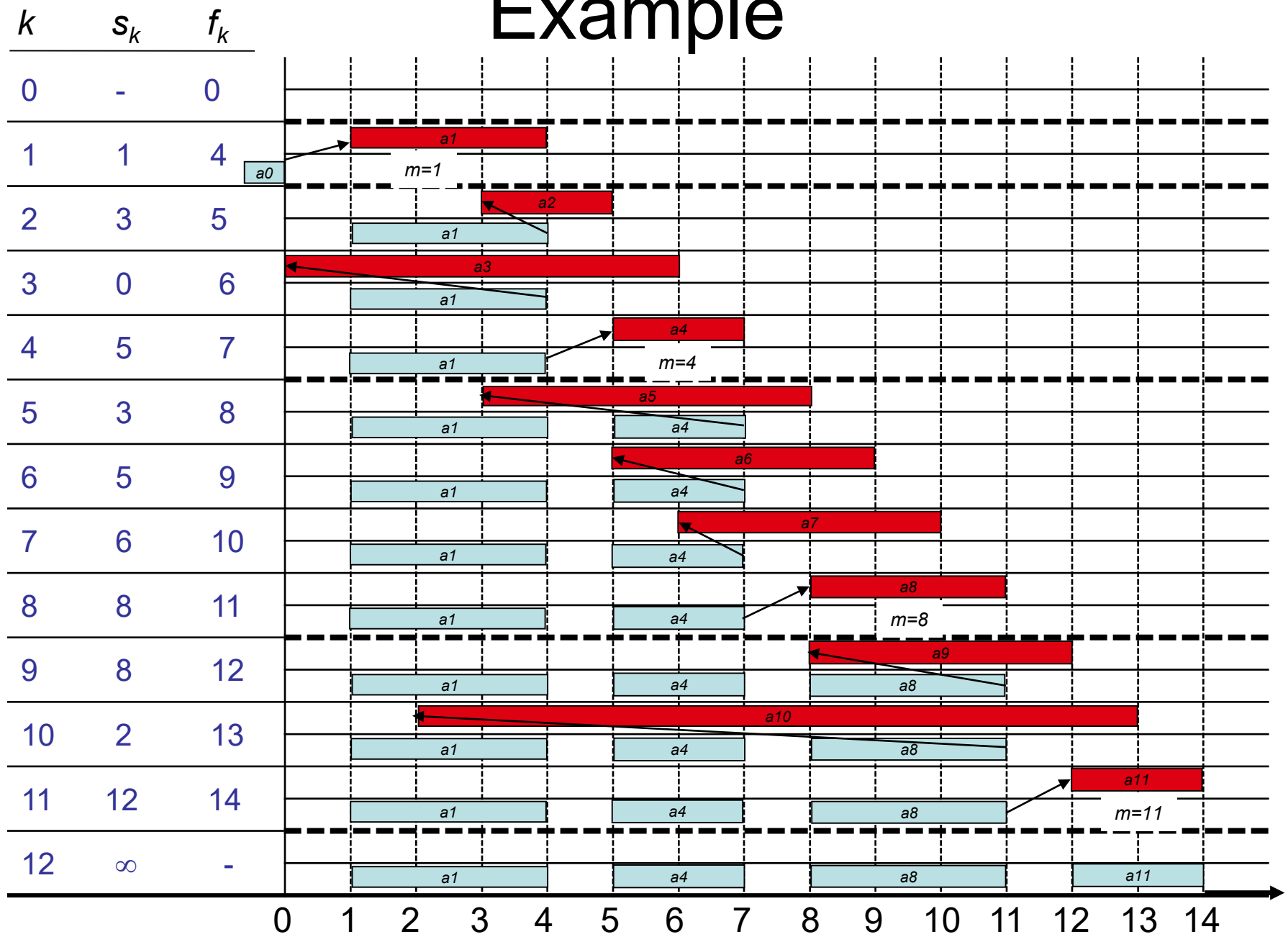
Characterizing the Subproblems

- The original problem: find the maximum subset of mutually compatible activities for $S = S_{0, n+1}$
- Activities are sorted by increasing finish time

$$a_0, a_1, a_2, a_3, \dots, a_{n+1}$$

- We always choose an activity with the earliest finish time
 - Greedy choice maximizes the unscheduled time remaining
 - Finish time of activities selected is strictly increasing

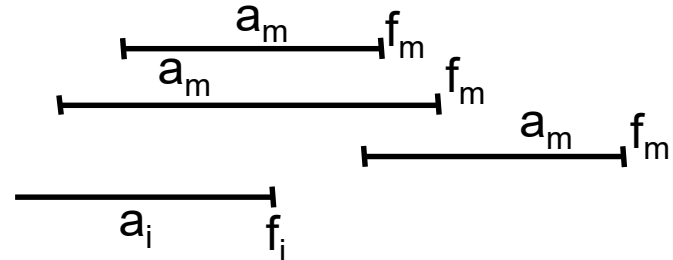
Example



An Incremental Algorithm

Alg.: GREEDY-ACTIVITY-SELECTOR(s, f, n)

1. $A \leftarrow \{a_1\}$
2. $i \leftarrow 1$
3. **for** $m \leftarrow 2$ **to** n
4. **do if** $s_m \geq f_i$ ▶ activity a_m is compatible with a_i
5. **then** $A \leftarrow A \cup \{a_m\}$
6. $i \leftarrow m$ ▶ a_i is most recent addition to A
7. **return** A



- Assumes that activities are ordered in increasing order of finish time
- Running time: $\Theta(n)$ – each activity is examined only once

Designing Greedy Algorithms

1. Cast the optimization problem as one for which:
we make a choice and are left with only one subproblem to solve
2. Prove that there is always an optimal solution to the original problem that makes the greedy choice
 - Making the greedy choice is always safe
3. Demonstrate that after making the greedy choice:
the greedy choice + an optimal solution to the resulting subproblem leads to an optimal solution

Correctness of Greedy Algorithms

1. Greedy Choice Property

- A globally optimal solution can be arrived at by making a locally optimal (greedy) choice

2. Optimal Substructure Property

- We know that we have arrived at a subproblem by making a greedy choice
- Optimal solution to subproblem + greedy choice $\Rightarrow$ optimal solution for the original problem

The Knapsack Problem

- **The 0-1 knapsack problem**

- A thief rubbing a store finds n items: the i -th item is worth v_i dollars and weights w_i pounds (v_i, w_i integers)
- The thief can only carry W pounds in his knapsack
- Items must be taken entirely or left behind
- Which items should the thief take to maximize the value of his load?

- **The fractional knapsack problem**

- Similar to above
- The thief can take fractions of items

Fractional Knapsack Problem

- Knapsack capacity: W
- There are n items: the i -th item has value v_i and weight w_i
- Goal:
 - find x_i such that for all $0 \leq x_i \leq 1, i = 1, 2, \dots, n$
 $\sum w_i x_i \leq W$ and
 $\sum x_i v_i$ is maximum

Fractional Knapsack Problem

Alg.: Fractional-Knapsack ($W, v[n], w[n]$)

1. While $w > 0$ and as long as there are items remaining
 2. pick item with maximum v_i/w_i
 3. $x_i \leftarrow \min(1, w/w_i)$
 4. remove item i from list
 5. $w \leftarrow w - x_i w_i$
- w – the amount of space remaining in the knapsack ($w = W$)
 - Running time: $\Theta(n)$ if items already ordered; else $\Theta(n \lg n)$

Optimal Substructure

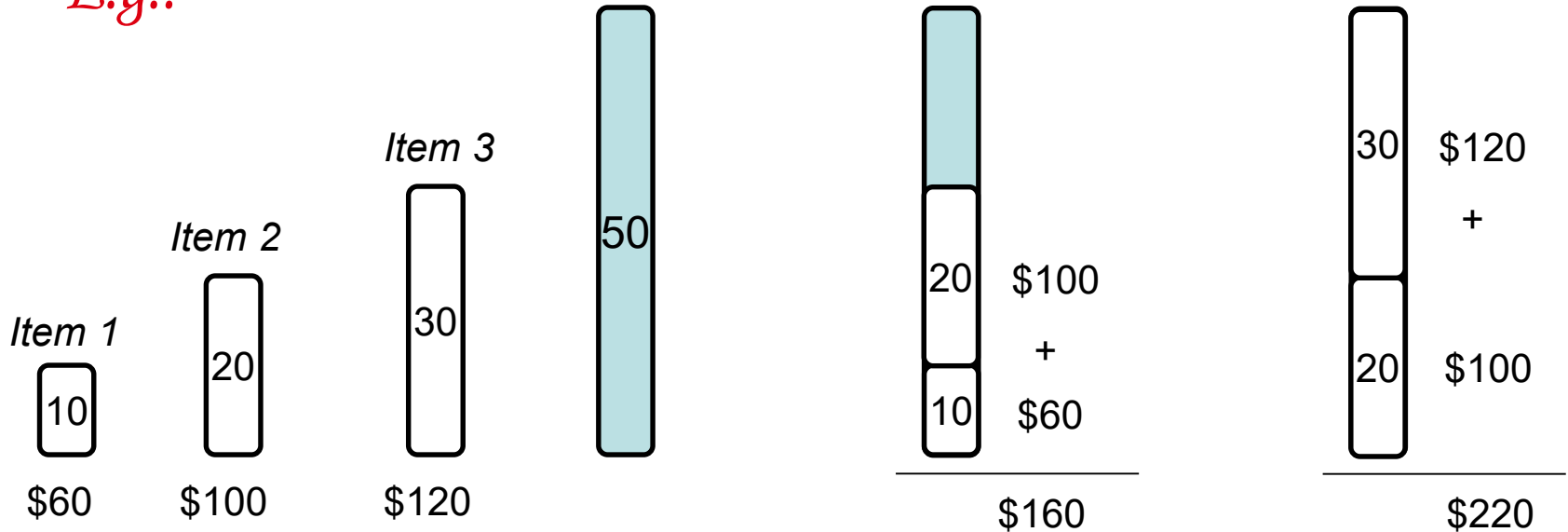
- Consider the most valuable load that weights at most W pounds
 - If we remove a weight w of item j from the optimal load
- ⇒ The remaining load must be the most valuable load weighing at most $W - w$ that can be taken from the remaining $n - 1$ items plus $w_j - w$ pounds of item j

The 0-1 Knapsack Problem

- Thief has a knapsack of capacity W
- There are n items: for i -th item value v_i and weight w_i
- Goal:
 - find x_i such that for all $x_i = \{0, 1\}$, $i = 1, 2, \dots, n$
 $\sum w_i x_i \leq W$ and
 $\sum x_i v_i$ is maximum

0-1 Knapsack - Greedy Strategy

• *E.g.:*



\$6/pound \$5/pound \$4/pound

- None of the solutions involving the greedy choice (item 1) leads to an optimal solution
 - The greedy choice property does not hold

0-1 Knapsack - Dynamic Programming

- $P(i, w)$ – the maximum profit that can be obtained from items 1 to i , if the knapsack has size w

- Case 1: thief takes item i

$$P(i, w) = v_i + P(i - 1, w - w_i)$$

- Case 2: thief does not take item i

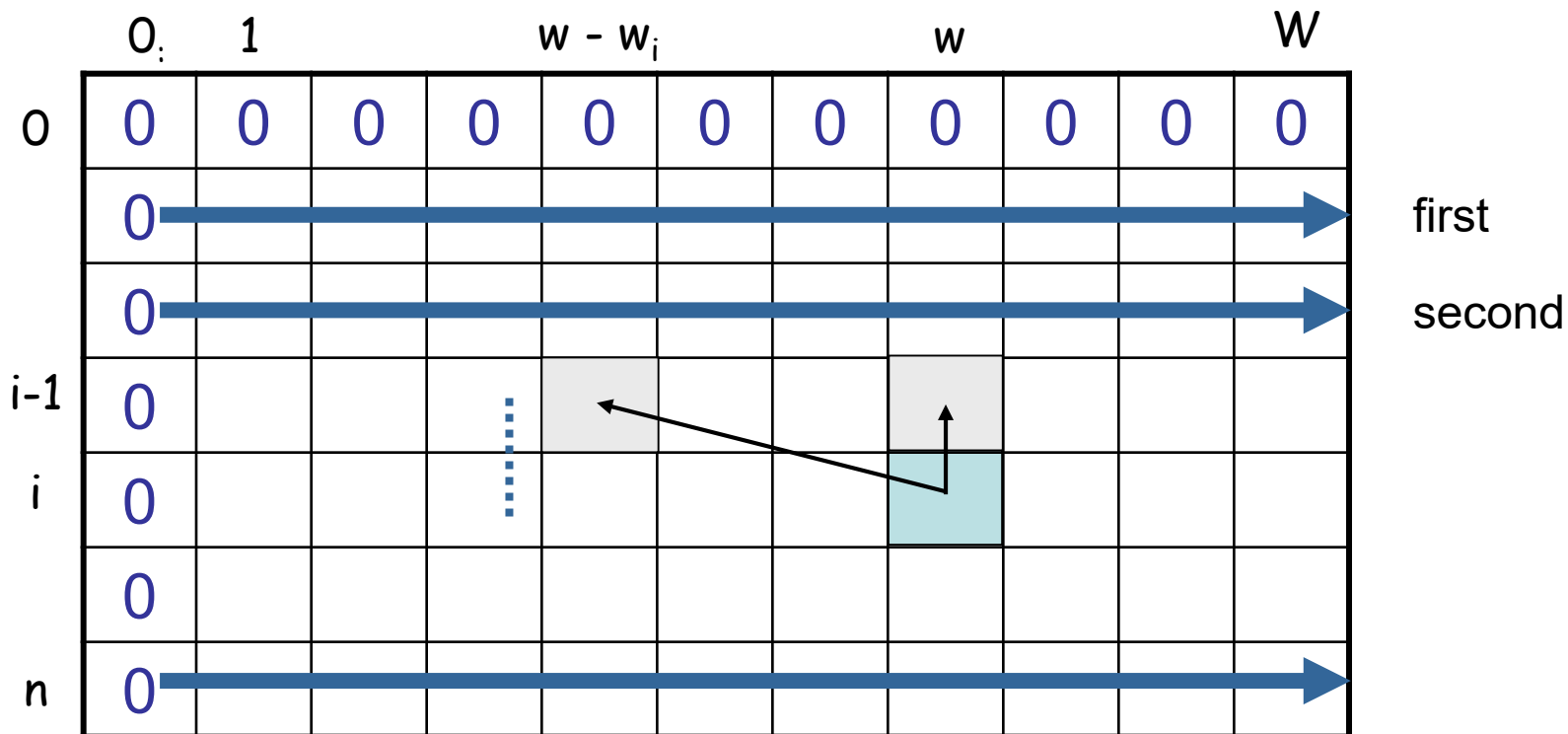
$$P(i, w) = P(i - 1, w)$$

0-1 Knapsack - Dynamic Programming

Item i was taken

Item i was not taken

$$P(i, w) = \max \{v_i + P(i - 1, w - w_i), P(i - 1, w) \}$$



Example:

$$P(i, w) = \max \{v_i + P(i - 1, w-w_i), P(i - 1, w) \}$$

W = 5

Item	Weight	Value
1	2	12
2	1	10
3	3	20
4	2	15

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

$$P(1, 1) = P(0, 1) = 0$$

$$P(1, 2) = \max\{12+0, 0\} = 12$$

$$P(1, 3) = \max\{12+0, 0\} = 12$$

$$P(1, 4) = \max\{12+0, 0\} = 12$$

$$P(1, 5) = \max\{12+0, 0\} = 12$$

$$P(2, 1) = \max\{10+0, 0\} = 10$$

$$P(2, 2) = \max\{10+0, 12\} = 12$$

$$P(2, 3) = \max\{10+12, 12\} = 22$$

$$P(2, 4) = \max\{10+12, 12\} = 22$$

$$P(2, 5) = \max\{10+12, 12\} = 22$$

$$P(3, 1) = P(2, 1) = 10$$

$$P(3, 2) = P(2, 2) = 12$$

$$P(3, 3) = \max\{20+0, 22\} = 22$$

$$P(3, 4) = \max\{20+10, 22\} = 30$$

$$P(4, 5) = \max\{20+12, 22\} = 32$$

$$P(4, 1) = P(3, 1) = 10$$

$$P(4, 2) = \max\{15+0, 12\} = 15$$

$$P(4, 3) = \max\{15+10, 22\} = 25$$

$$P(4, 4) = \max\{15+12, 30\} = 30$$

$$P(4, 5) = \max\{15+22, 32\} = 37$$

Reconstructing the Optimal Solution

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

- Item 4
- Item 2
- Item 1

- Start at $P(n, W)$
- When you go left-up $\Rightarrow$ item i has been taken
- When you go straight up $\Rightarrow$ item i has not been taken

Huffman Codes



- Idea:
 - Use the frequencies of occurrence of characters to build an optimal way of representing each character

	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5

- ***Binary character code***
 - Uniquely represents a character by a binary string



Fixed-Length Codes



E.g.: Data file containing 100,000 characters

	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5

- 3 bits needed
- $a = 000$, $b = 001$, $c = 010$, $d = 011$, $e = 100$, $f = 101$
- Requires: $100,000 \cdot 3 = 300,000$ bits



Variable-Length Codes



E.g.: Data file containing 100,000 characters

	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5

- Assign short codewords to frequent characters and long codewords to infrequent characters
- $a = 0$, $b = 101$, $c = 100$, $d = 111$, $e = 1101$, $f = 1100$
- $(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000$
 $= 224,000$ bits



Prefix Codes

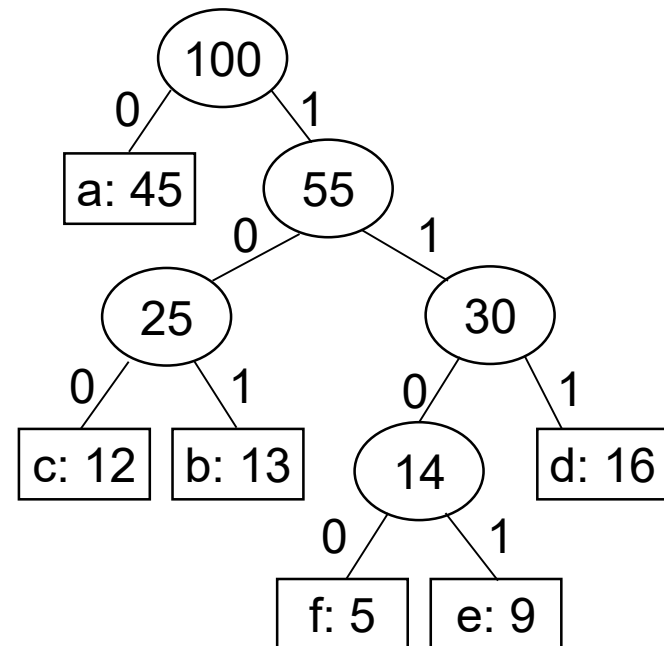
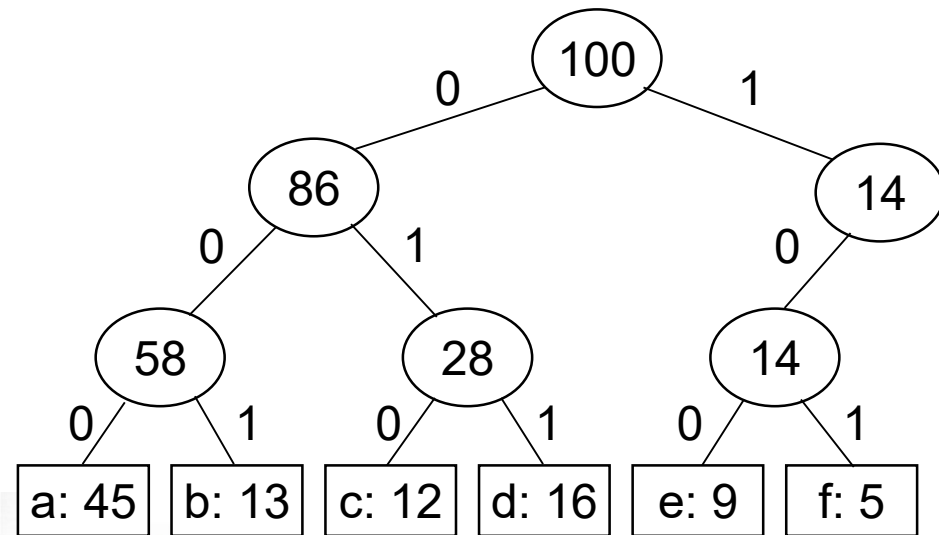


- Prefix codes:
 - Codes for which no codeword is also a prefix of some other codeword
 - Better name would be “prefix-free codes”
- We can achieve optimal data compression using prefix codes
 - We will restrict our attention to prefix codes



Prefix Code Representation

- Binary tree whose leaves are the given characters
- Binary codeword
 - the path from the root to the character, where 0 means “go to the left child” and 1 means “go to the right child”
- Length of the codeword
 - Length of the path from root to the character leaf (depth of node)

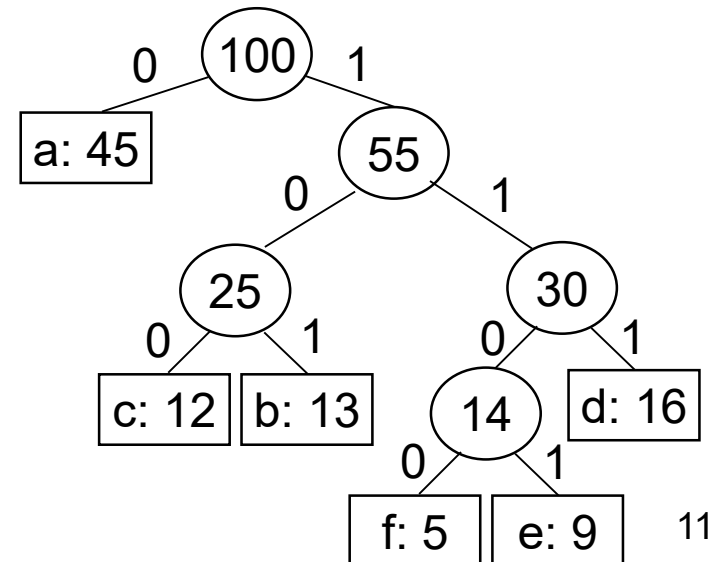
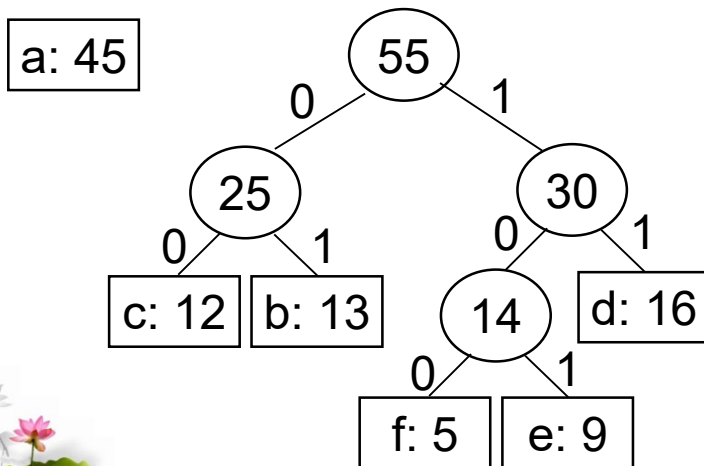
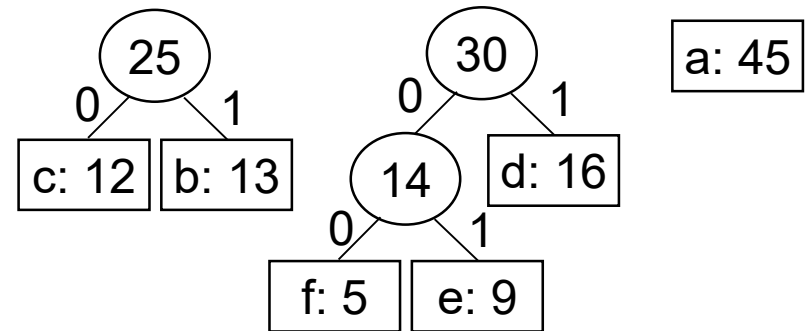
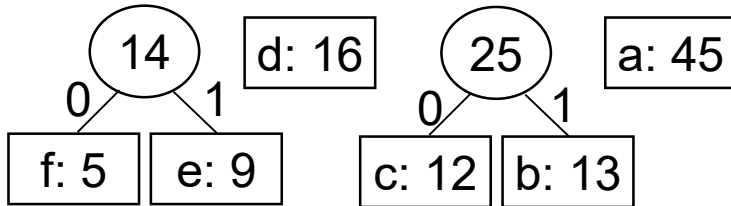


Example



f: 5 e: 9 c: 12 b: 13 d: 16 a: 45

c: 12 b: 13 d: 16 a: 45



Building a Huffman Code



Alg.: HUFFMAN($\mathcal{C}$)

Running time: $O(n \lg n)$

1. $n \leftarrow |\mathcal{C}|$

2. $Q \leftarrow \mathcal{C}$ $\longleftarrow O(n)$

3. **for** $i \leftarrow 1$ **to** $n - 1$

4. **do** allocate a new node z

5. $\text{left}[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$

6. $\text{right}[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$

7. $f[z] \leftarrow f[x] + f[y]$

8. INSERT (Q, z)

9. **return** EXTRACT-MIN(Q)

$O(n \lg n)$



Basic Graph Algorithms

Graphs

- A graph $G = (V, E)$
 - V = set of vertices
 - E = set of edges = subset of $V \times V$
 - Thus $|E| = O(|V|^2)$

Graph Variations

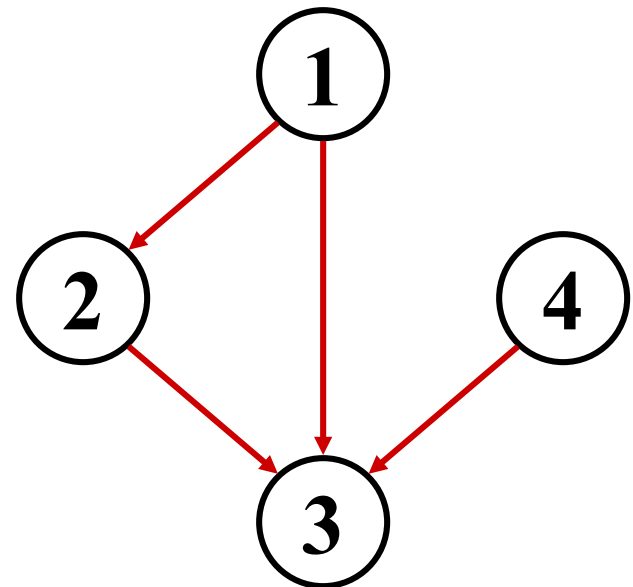
- Variations:
 - A *connected graph* has a path from every vertex to every other
 - In an *undirected graph*:
 - Edge (u,v) = edge (v,u)
 - No self-loops
 - In a *directed graph*:
 - Edge (u,v) goes from vertex u to vertex v , notated $u \rightarrow v$

Representing Graphs

- Assume $V = \{1, 2, \dots, n\}$
- An *adjacency matrix* represents the graph as a $n \times n$ matrix A :
 - $A[i, j] = 1$ if edge $(i, j) \in E$ (or weight of edge)
 $= 0$ if edge $(i, j) \notin E$

Graphs: Adjacency List

- Adjacency list: for each vertex $v \in V$, store a list of vertices adjacent to v
- Example:
 - $\text{Adj}[1] = \{2,3\}$
 - $\text{Adj}[2] = \{3\}$
 - $\text{Adj}[3] = \{\}$
 - $\text{Adj}[4] = \{3\}$
- Variation: can also keep a list of edges coming *into* vertex



Graphs: Adjacency List

- How much storage is required?
 - The *degree* of a vertex $v = \#$ incident edges
 - Directed graphs have in-degree, out-degree
 - For directed graphs, # of items in adjacency lists is
$$\sum \text{out-degree}(v) = |E|$$
takes $\Theta(V + E)$ storage (*Why?*)
 - For undirected graphs, # items in adj lists is
$$\sum \text{degree}(v) = 2 |E| \quad (\textit{handshaking lemma})$$
also $\Theta(V + E)$ storage
- So: Adjacency lists take $O(V+E)$ storage

Graph Searching

- Given: a graph $G = (V, E)$, directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
 - Pick a vertex as the root
 - Choose certain edges to produce a tree
 - Note: might also build a *forest* if graph is not connected

Breadth-First Search

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.

Breadth-First Search

- Again will associate vertex “colors” to guide the algorithm
 - White vertices have not been discovered
 - All vertices start out white
 - Grey vertices are discovered but not fully explored
 - They may be adjacent to white vertices
 - Black vertices are discovered and fully explored
 - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  Touch every vertex:  $O(V)$   
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  u = every vertex, but only once  
        for each v  $\in$  u->adj { (Why?)  
             So v = every vertex that appears in some other vert's adjacency list if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

What will be the running time?

Total running time: $O(V+E)$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
 - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s
 - Proof given in the book (p. 472-5)
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
 - Thus can use BFS to calculate shortest path from one vertex to another in $O(V+E)$ time

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
 - Explore “deeper” in the graph whenever possible
 - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
 - When all of v 's edges have been explored, backtrack to the vertex from which v was discovered

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->d represent?

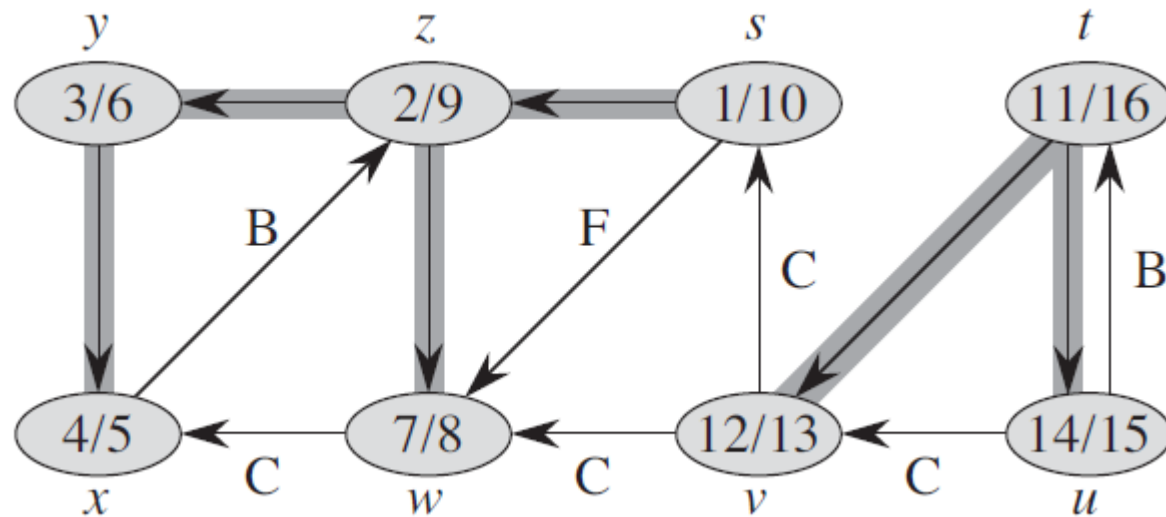
Depth-First Sort Analysis

- This running time argument is an informal example of *amortized analysis*
 - “Charge” the exploration of edge to the edge:
 - Each loop in DFS_Visit can be attributed to an edge in the graph
 - Runs once/edge if directed graph, twice if undirected
 - Thus loop will run in $O(E)$ time, algorithm $O(V+E)$
 - ◆ Considered linear for graph, b/c adj list requires $O(V+E)$ storage
 - Important to be comfortable with this kind of reasoning and analysis

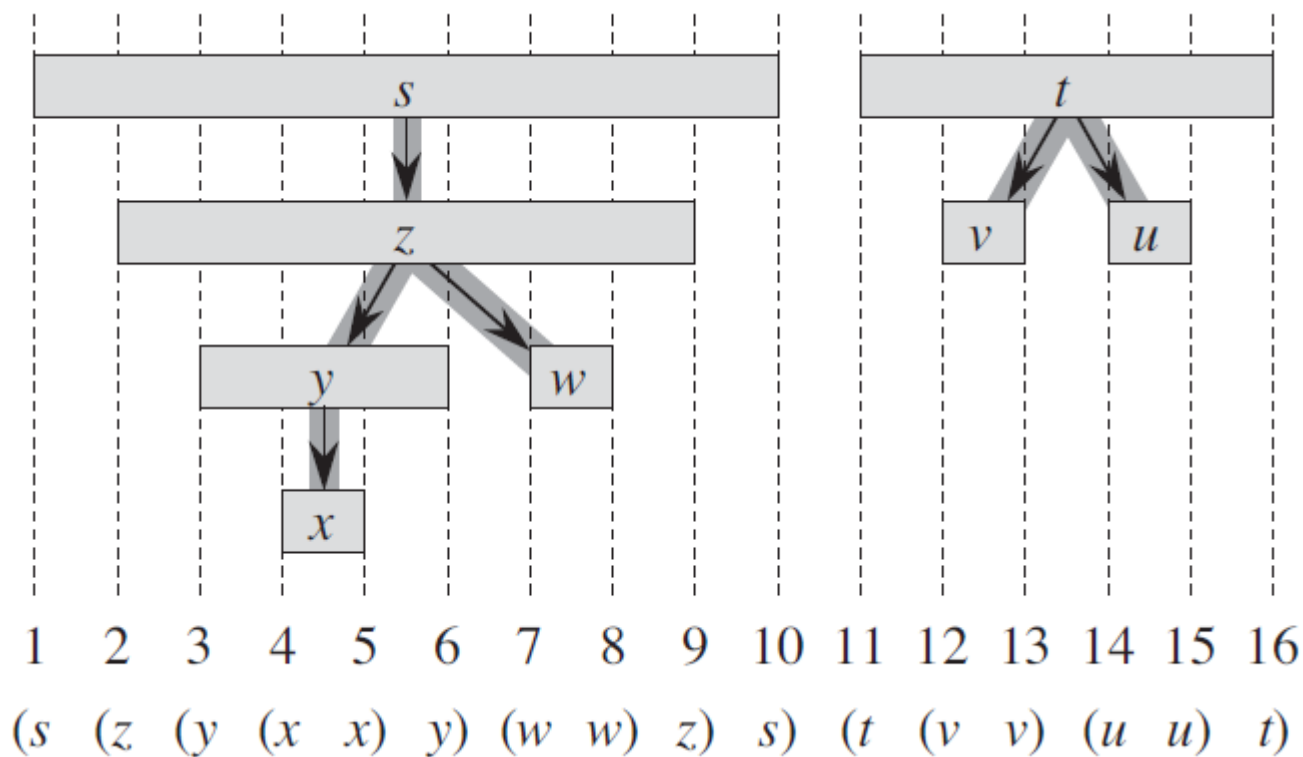
DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - *Cross edge*: between a tree or subtrees
- Note: tree & back edges are important; most algorithms don't distinguish forward & cross

(a)



(b)



Topological Sort

- *Topological sort* of a DAG:
 - Linear ordering of all vertices in graph G such that vertex u comes before vertex v if edge $(u, v) \in G$
- Real-world example: getting dressed

Topological Sort Algorithm

```
Topological-Sort()
```

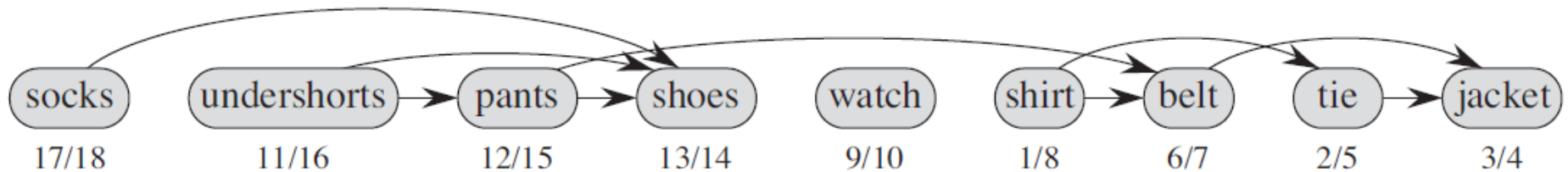
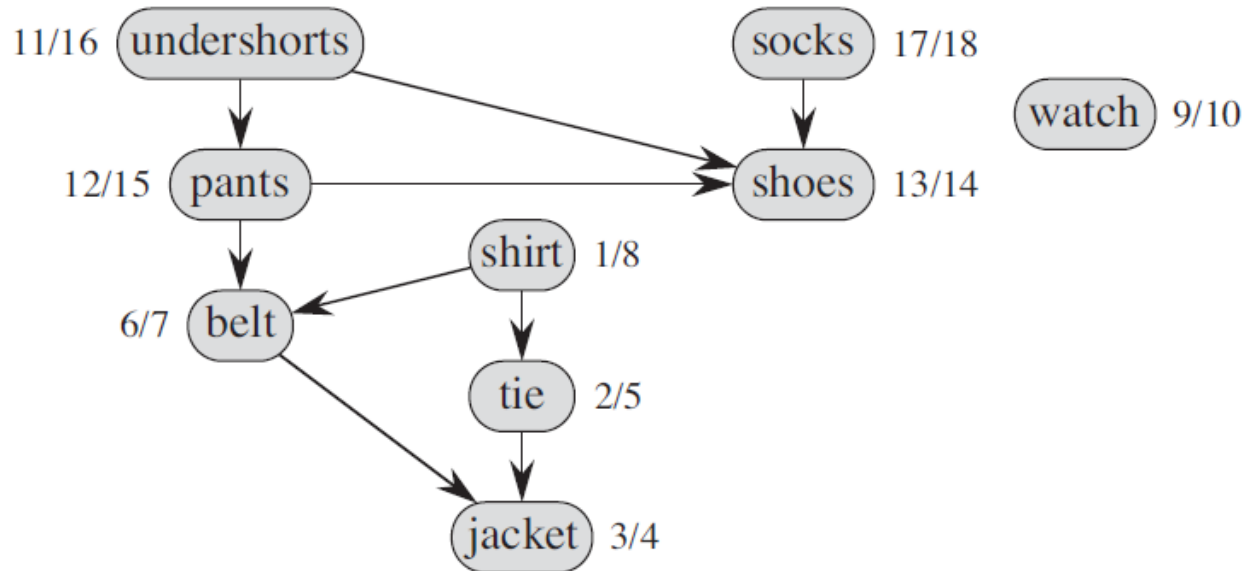
```
{  
    Run DFS  
    When a vertex is finished, output it  
    Vertices are output in reverse  
    topological order  
}
```

- Time: $O(V+E)$
- Correctness: Want to prove that
$$(u,v) \in G \Rightarrow u \rightarrow f > v \rightarrow f$$

Correctness of Topological Sort

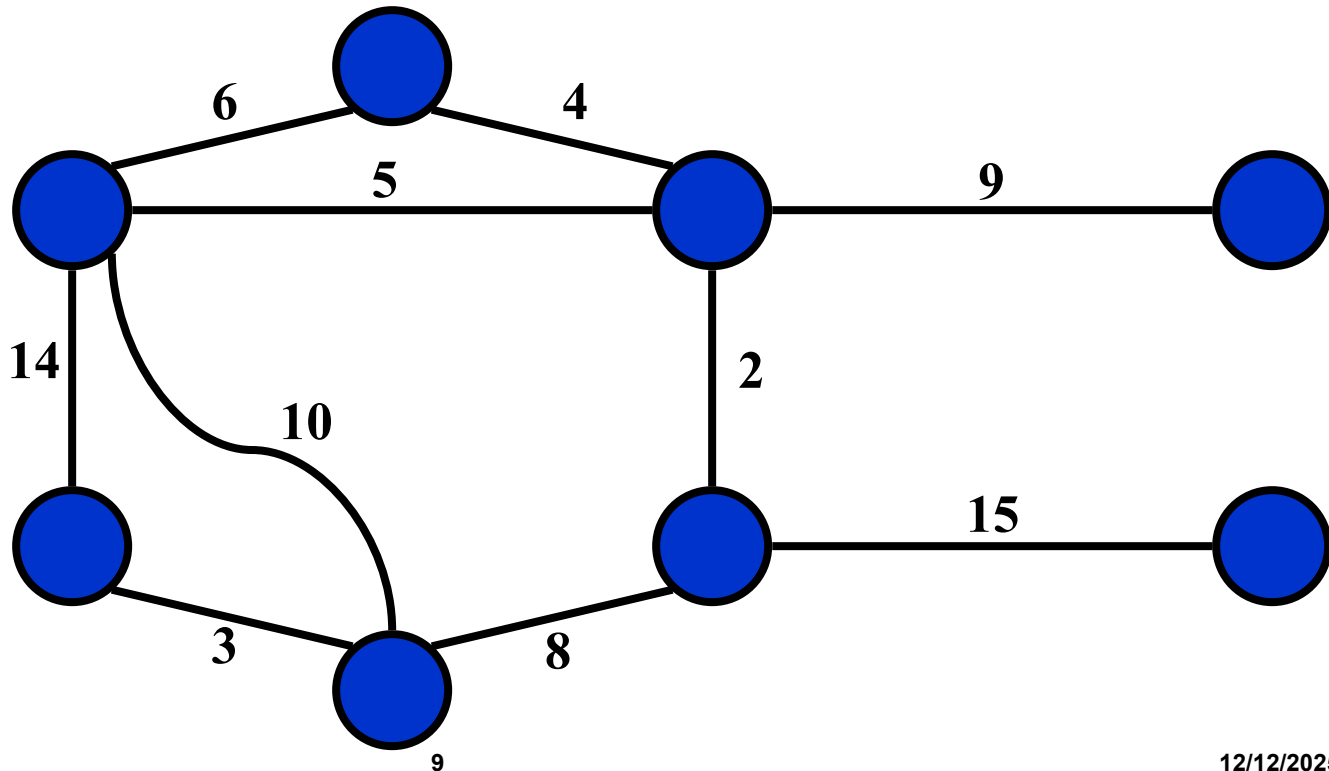
- Claim: $(u, v) \in G \Rightarrow u \rightarrow f > v \rightarrow f$
 - When (u, v) is explored, u is yellow
 - $v = \text{yellow} \Rightarrow (u, v)$ is back edge. Contradiction (*Why?*)
 - $v = \text{white} \Rightarrow v$ becomes descendent of $u \Rightarrow v \rightarrow f < u \rightarrow f$
(since must finish v before backtracking and finishing u)
 - $v = \text{black} \Rightarrow v$ already finished $\Rightarrow v \rightarrow f < u \rightarrow f$

Getting Dressed



Minimum Spanning Tree

- Problem: given a connected, undirected, weighted graph, find a *spanning tree* using edges that minimize the total weight



Minimum Spanning Tree

- MSTs satisfy the *optimal substructure* property: an optimal tree is composed of optimal subtrees
 - Let T be an MST of G with an edge (u, v) in the middle
 - Removing (u, v) partitions T into two trees T_1 and T_2
 - Claim: T_1 is an MST of $G_1 = (V_1, E_1)$, and T_2 is an MST of $G_2 = (V_2, E_2)$ (*Do V_1 and V_2 share vertices? Why?*)
 - Proof: $w(T) = w(u, v) + w(T_1) + w(T_2)$
(There can't be a better tree than T_1 or T_2 , or T would be suboptimal)

Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

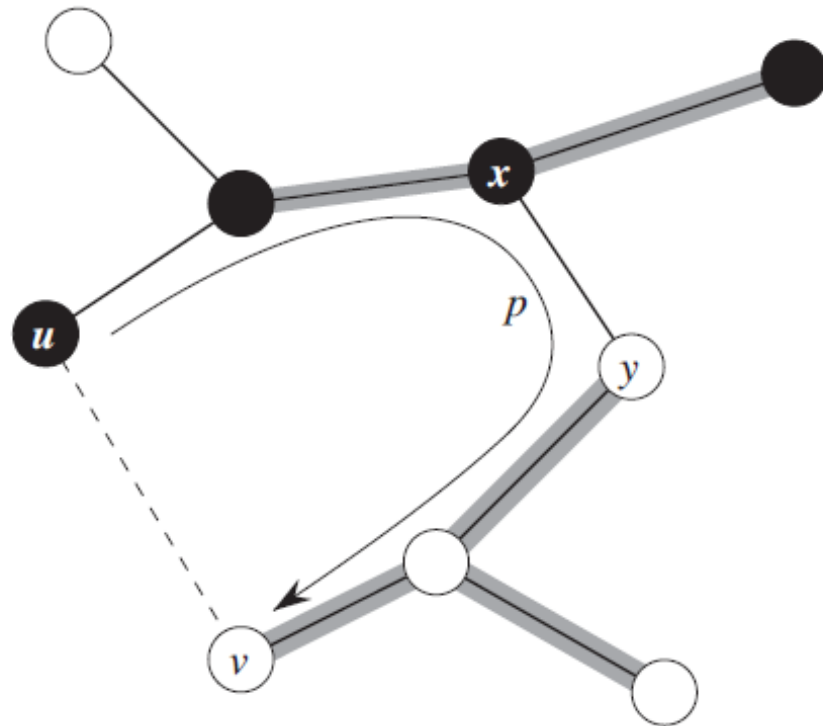
Initialization: After line 1, the set A trivially satisfies the loop invariant.

Maintenance: The loop in lines 2–4 maintains the invariant by adding only safe edges.

Termination: All edges added to A are in a minimum spanning tree, and so the set A returned in line 5 must be a minimum spanning tree.

Minimum Spanning Tree

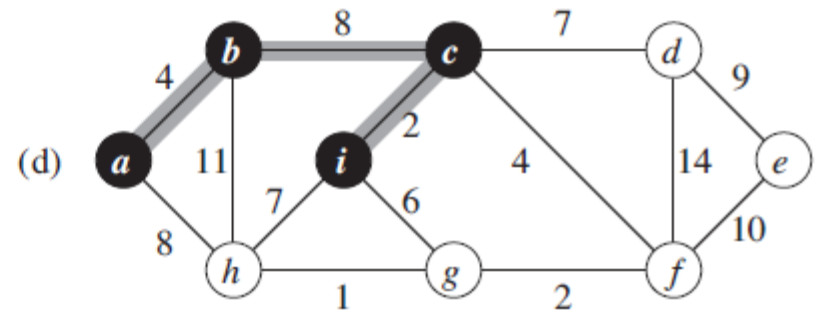
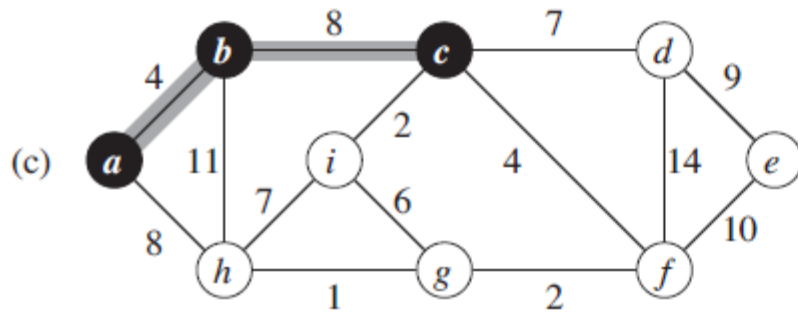
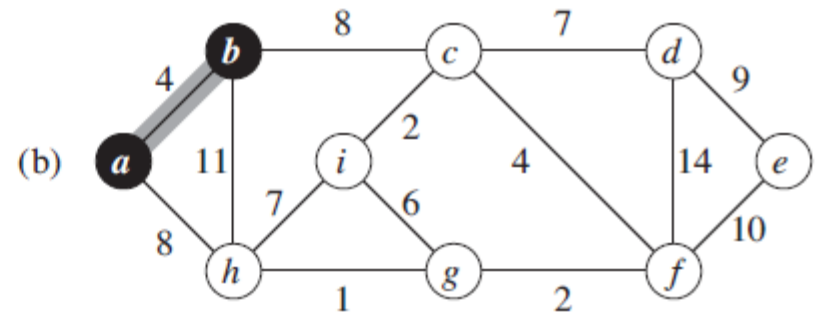
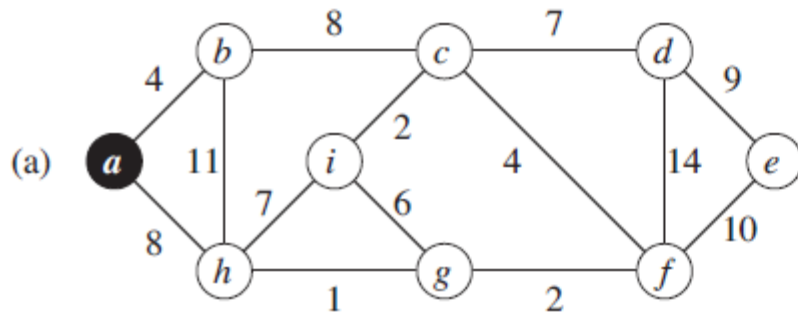
- Thm 23.1 :
 - Let T be MST of G , and let $A \subseteq T$ be subtree of T
 - Let (u,v) be min-weight edge connecting A to $V-A$
 - Then $(u,v) \in T$
- Proof:
By exchange..

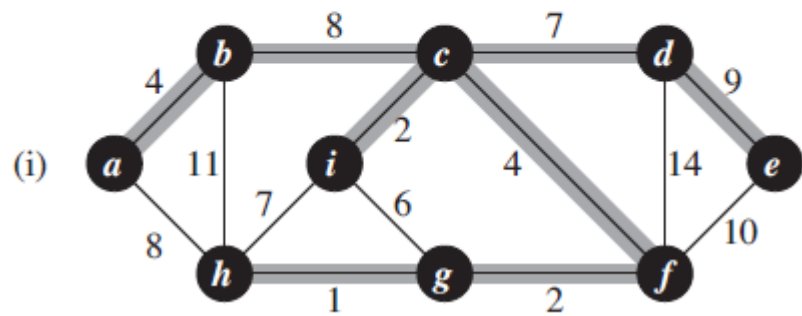
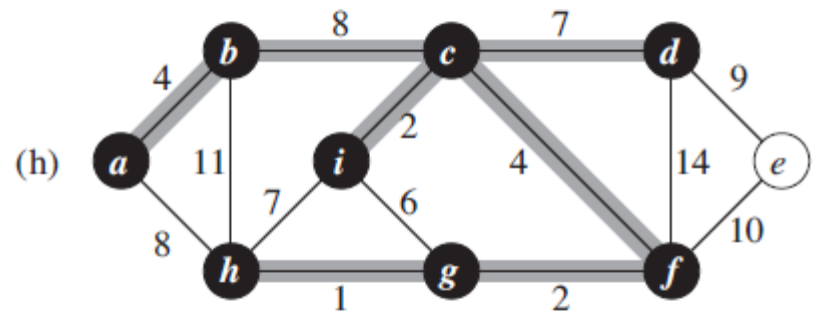
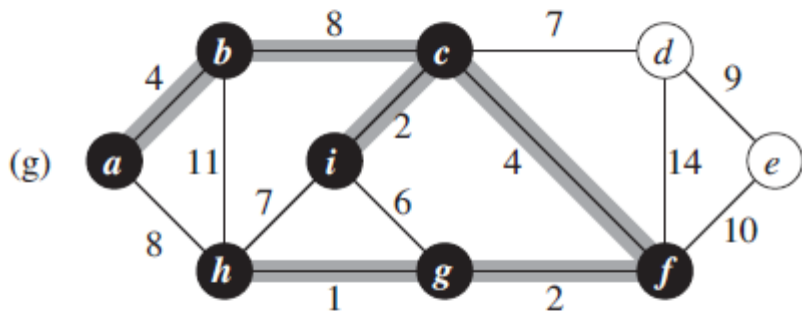
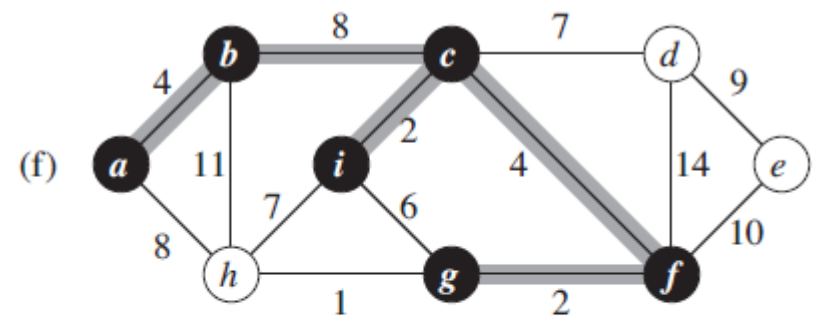
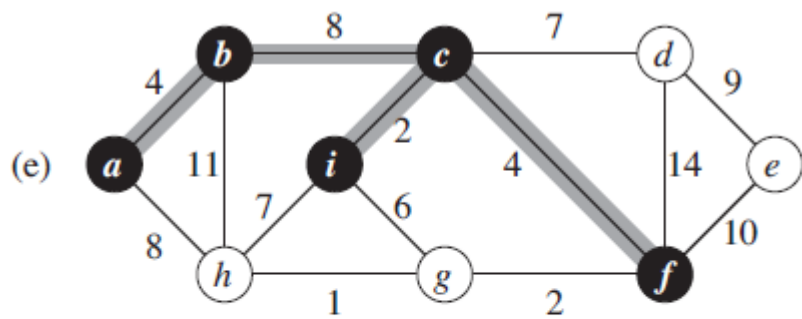


Prim's Algorithm

```
MST-Prim( $G, w, r$ )
   $Q = V[G];$ 
  for each  $u \in Q$ 
     $key[u] = \infty;$ 
   $key[r] = 0;$ 
   $p[r] = \text{NULL};$ 
  while ( $Q$  not empty)
     $u = \text{ExtractMin}(Q);$ 
    for each  $v \in \text{Adj}[u]$ 
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
         $p[v] = u;$ 
         $key[v] = w(u, v);$ 
```

Prim's Algorithm





Analysis: Prim's Algorithm

MST-Prim(G, w, r)

$Q = V[G];$

for each $u \in Q$

$key[u] = \infty;$

$key[r] = 0;$

$p[r] = \text{NULL};$

while (Q not empty)

$u = \text{ExtractMin}(Q);$

 for each $v \in \text{Adj}[u]$

 if ($v \in Q$ and $w(u, v) < key[v]$)

$p[v] = u;$

$key[v] = w(u, v);$

What will be the running time?

A: Depends on queue

binary heap: $O(E \lg V)$

Fibonacci heap: $O(V \lg V + E)$

Kruskal's Algorithm

- Starts with each vertex in its own component.
- **Repeatedly merges two components** into one by choosing a light edge that connects them (i.e., a light edge crossing the cut between them).
- Scans the set of edges in monotonically increasing order by weight.
- Uses a **disjoint-set data structure** to determine whether an edge connects vertices in different components.

Disjoint Set Data Structure

- Want to maintain a collection $S = \{S_1, \dots, S_k\}$ of **disjoint dynamic sets**.
- Each set has a **representative member**.
- **Operations:**
 - **Make-Set(x)**: Make new singleton set containing object x (x is representative).
 - **Union(x, y)**: Unite sets containing x and y into a new set that is a union of the two sets. (Representative is any member of the union.)
 - **Find-Set(x)**: Returns pointer to representative of set containing x .
- **Complexity:** $O(m \alpha(n))$
 - n = no. of Make-Set operations, m = total no. of operations.
 - **Note:** $m \geq n$.
 - Suffices to know $\alpha(n) = O(\lg^* n) = O(\lg n)$.
 $\lg^*(n) = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$

Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

- Initialization -- $O(V)$.
- Sort Edges -- $O(E \lg E)$.

- Set Operations --
 $O((V+E)\alpha(V))$
- Total is $O(E \lg E)$.

- $E = O(V^2)$
- Therefore, $\lg E = O(\lg V)$
- Hence, complexity is
 $O(E \lg V)$.

Chapter 24

Single-Source Shortest Paths

Shortest Path Problems

- **Input:**

- Directed graph $G = (V, E)$
- Weight function $w : E \rightarrow \mathbf{R}$

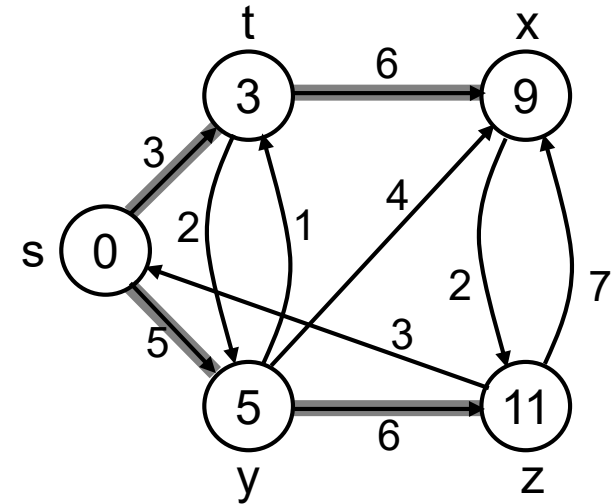
- **Weight of path $p = \langle v_0, v_1, \dots, v_k \rangle$**

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

- **Shortest-path weight from u to v :**

$$\delta(u, v) = \begin{cases} \min \{ w(p) : u \xrightarrow{p} v \} & \text{if there exists a path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

- Shortest path u to v is any path p such that $w(p) = \delta(u, v)$



Variants of Shortest Paths

- **Single-source shortest path**

- $G = (V, E) \Rightarrow$ find a shortest path from a given source vertex s to each vertex $v \in V$

- **Single-destination shortest path**

- Find a shortest path to a given destination vertex t from each vertex v
- Reverse the direction of each edge $\Rightarrow$ single-source

- **Single-pair shortest path**

- Find a shortest path from u to v for given vertices u and v
- Solve the single-source problem

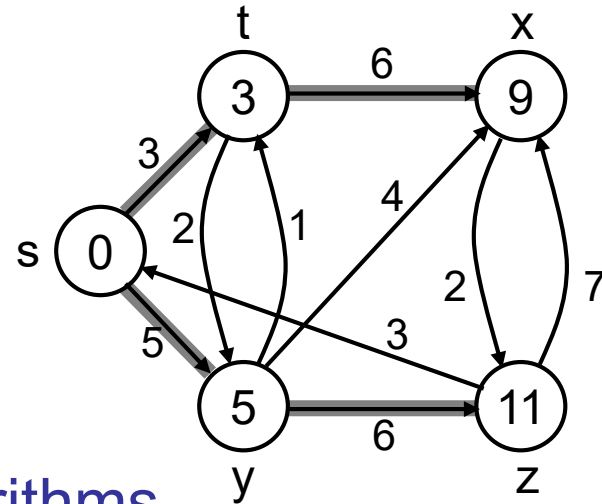
- **All-pairs shortest-paths**

- Find a shortest path from u to v for every pair of vertices u and v

Computing Shortest-Paths

For each vertex $v \in V$ maintain:

- $d[v] = \delta(s, v)$: a **shortest-path estimate**
 - Initially, $d[v] = \infty$
 - Reduces as algorithms progress
- $\pi[v] =$ **predecessor** of v on a shortest path from s
- All the single-source shortest-paths algorithms
 - Initialize $d[v]$ and $\pi[v]$
 - Then relax edges
- The algorithms differ in the order and how many times they relax each edge



Bellman-Ford Algorithm

- Single-source shortest paths problem
 - Computes $d[v]$ and $\pi[v]$ for all $v \in V$
- Allows negative edge weights
- Returns:
 - TRUE if no negative-weight cycles are reachable from the source s
 - FALSE otherwise $\Rightarrow$ no solution exists
- Idea:
 - Traverse all the edges $|V - 1|$ times, every time performing a relaxation step of each edge

Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$                                  $\Theta(V)$ 
3      for each edge  $(u, v) \in G.E$                              $O(E)$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$                                  $O(E)$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

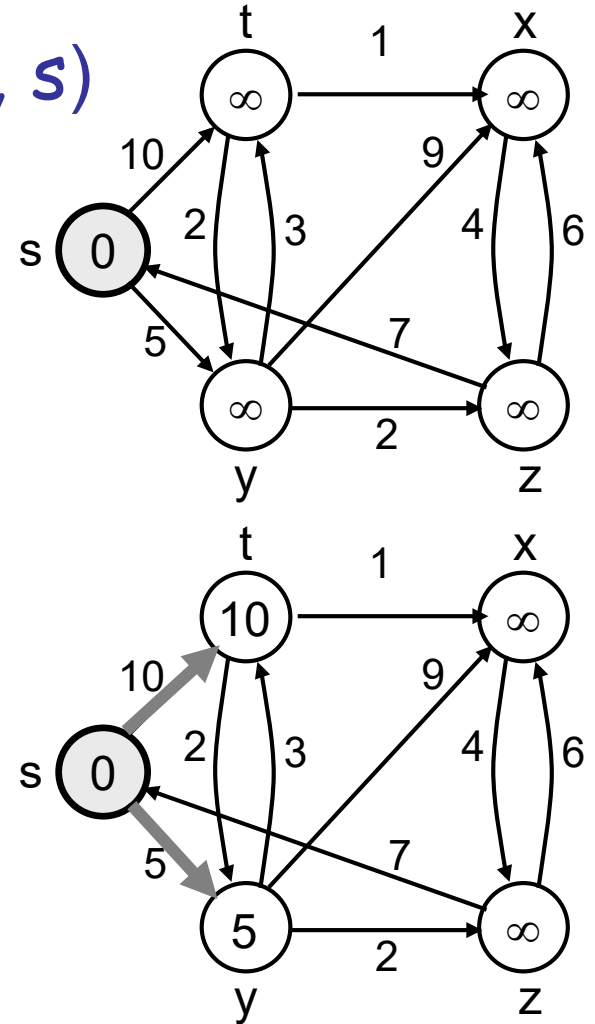
$O(VE)$

Dijkstra's Algorithm

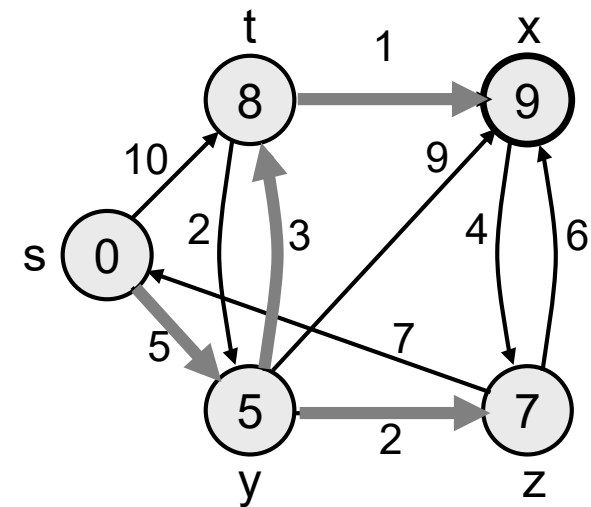
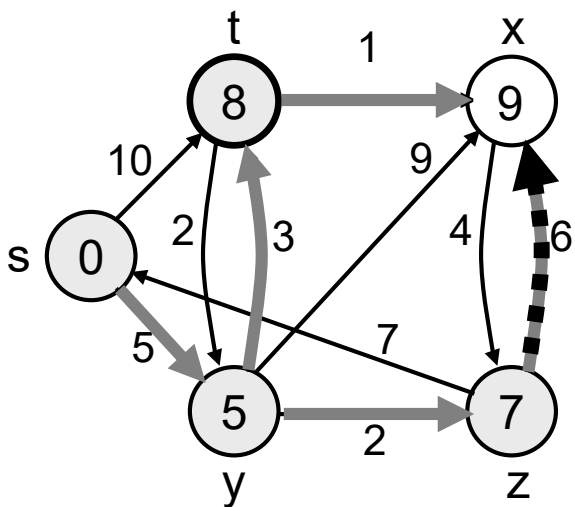
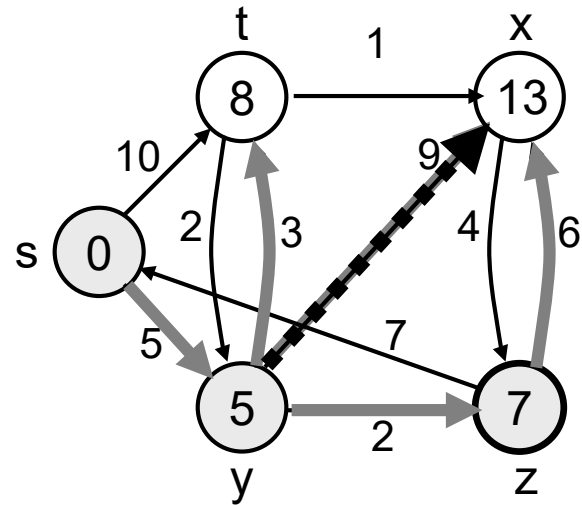
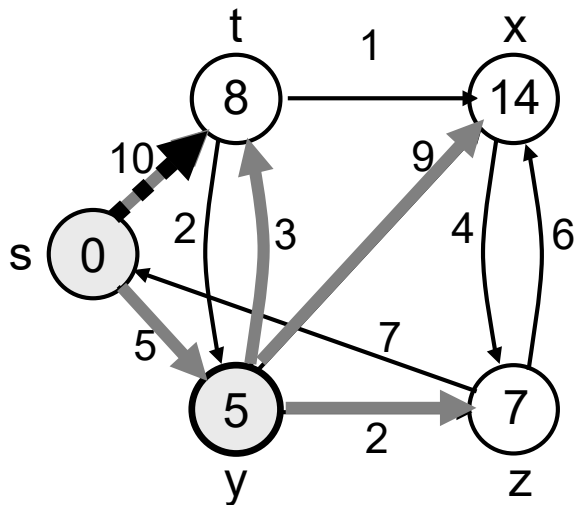
- Single-source shortest path problem:
 - No negative-weight edges: $w(u, v) > 0 \ \forall \ (u, v) \in E$
- Maintains two sets of vertices:
 - S = vertices whose final shortest-path weights have already been determined
 - Q = vertices in $V - S$: min-priority queue
 - Keys in Q are estimates of shortest-path weights ($d[v]$)
- Repeatedly select a vertex $u \in V - S$, with the minimum shortest-path estimate $d[v]$

Dijkstra (G, w, s)

1. INITIALIZE-SINGLE-SOURCE(V, s)
2. $S \leftarrow \emptyset$
3. $Q \leftarrow V[G]$
4. **while** $Q \neq \emptyset$
5. **do** $u \leftarrow \text{EXTRACT-MIN}(Q)$
6. $S \leftarrow S \cup \{u\}$
7. **for** each vertex $v \in \text{Adj}[u]$
8. **do** RELAX(u, v, w)



Example

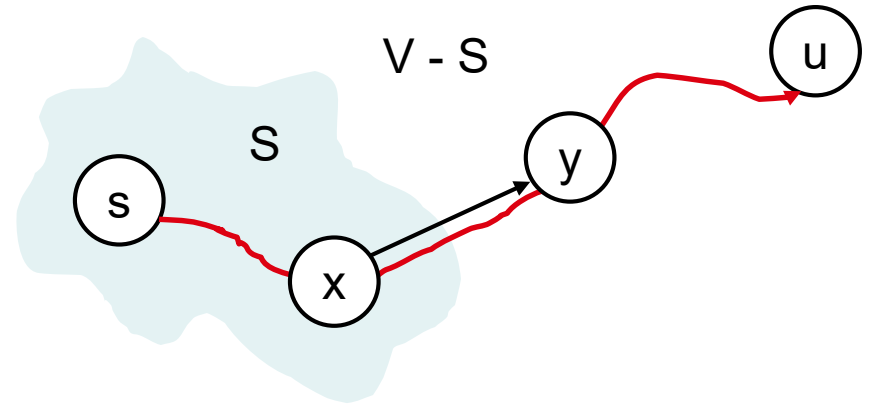


Correctness of Dijkstra's Algorithm

- For each vertex $u \in V$, we have $d[u] = \delta(s, u)$ at the time when u is added to S .

Proof:

- Let u be the first vertex for which $d[u] \neq \delta(s, u)$ when added to S
- Let's look at a true shortest path from s to u (p):
 - Before adding u to S , path p connects a vertex in S (s) with one in $V - S$ (u)
 - Let (x, y) be the first edge crossing $(S, V - S)$



Correctness of Dijkstra's Algorithm

- Claim: $d[y] = \delta(s, y)$ at the time when u is added to S

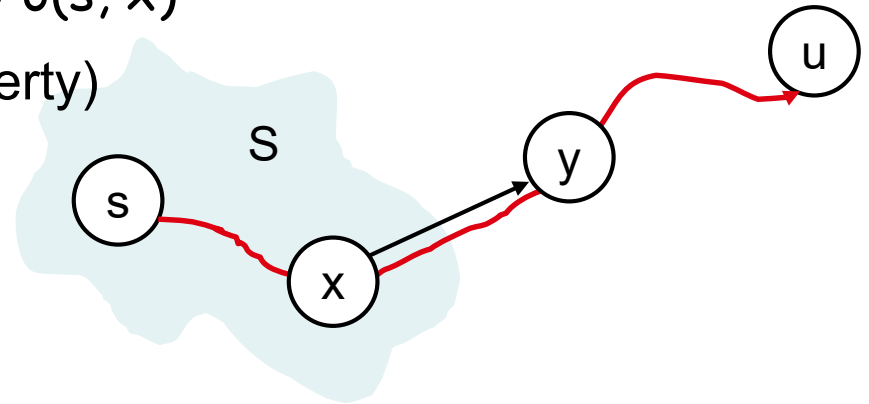
- u is the first vertex with $d[u] \neq \delta(s, u)$
- $x \in S$ and we must have $d[x] = \delta(s, x)$
- Relax (x, y) (convergence property)
 $\Rightarrow d[y] = \delta(s, y)$

- y is before u on the shortest path from s to u :

$$\Rightarrow d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]$$

- Both u and $y \in V - S$ when u was chosen and since we chose u : $d[u] \leq d[y] \Rightarrow$ relation above is an equality:

$$d[y] = \delta(s, y) = \delta(s, u) = d[u] \text{ contradiction with choice of } u!$$



Time complexity

1. INITIALIZE-SINGLE-SOURCE(V, s) $\leftarrow \Theta(V)$
2. $S \leftarrow \emptyset$
3. $Q \leftarrow V[G] \leftarrow O(V)$ build min-heap
4. **while** $Q \neq \emptyset$ $\leftarrow$ Executed $O(V)$ times
5. **do** $u \leftarrow \text{EXTRACT-MIN}(Q) \leftarrow O(\lg V)$
6. $S \leftarrow S \cup \{u\}$
7. **for** each vertex $v \in \text{Adj}[u]$
8. **do** RELAX(u, v, w) $\leftarrow O(E)$ times; $O(\lg V)$

Running time: $O(V \lg V + E \lg V) = O(E \lg V)$